

MegaMOO Developer Manual

MegaMOO Developer Manual

MegaMOO is a **scratch-built native-Python codebase** in the tradition of [LambdaMOO](#) — a persistent, multiplayer, fully programmable text world, **not a line by line port of the C LambdaMOO source**. The MegaMOO server is an original implementation that adopts the MOO paradigm — numbered objects, single-parent prototype inheritance, natural language parsing and on-the-fly verb programming live inside the game — realized idiomatically in roughly **1.4 MB** of modern Python — server, entry point, and verb library — built on `asyncio` with **zero third-party dependencies**.

This manual is written for developers — people who will **build worlds, program verbs**, or **operate the server** — rather than for players. It explains not just *how* to use each subsystem but *why* it is built the way it is, with references into the source so you can follow the engine end to end.

The single defining idea: in a classic MOO you program in a purpose-built MOO # > language. In MegaMOO, **the in-world programming language is Python itself**. Every verb — from `look` to `@dig` — is a plain `.py` file with the full standard library available, executed live against a persistent object database.

How to read this manual

Document	Audience	What it covers
01 — Architecture	Everyone	The object model, the request lifecycle from socket to verb to disk, persistence, and the design philosophy. Start here.
02 — Writing Verbs	Verb authors / programmers	The injected namespace, the source preprocessor, <code>\$</code> constants, matching, custom parsers (verb types), emit substitution, the task system. The heart of the engine.
03 — Building Worlds	Builders / staff	The 64-command <code>@</code> -toolkit: rooms, exits, objects, descriptions, properties, and in-game verb programming, with a worked example.
04 — Operations	Operators	Running the server, command-line flags, configuration, the JSON API and MCP integration, persistence/backups, and the permission ladder.
05 — Engine Systems	Verb authors / programmers	The runtime systems verbs plug into: hooks (lifecycle events), the ticker (scheduled verbs), and the effects system.
06 — The Prototype Library	Builders / programmers	The shipped base objects — rooms (+ the compass), exits, containers, furniture, wearables, consumables — and their properties and verbs.
07 — The Help System	Builders / staff	The in-game <code>help</code> command: its resolution ladder, the <code>#34</code> topic store (articles and categories), self-documenting verb docstrings, and object introspection (<code>help #obj</code>).

The engine at a glance

Subsystem	Module	Responsibility
Server core	<code>moo/server.py</code>	Event loop, single-threaded verb execution via a one-worker executor, context propagation, graceful shutdown/restart
Networking	<code>moo/network.py</code>	Telnet/WebSocket connections, protocol negotiation, color and wrapping
Database	<code>moo/database.py</code>	SQLite persistence, lazy object cache, checkpointing
Object model	<code>moo/objects.py</code>	Inheritance, properties as native Python attributes, flags, tags
Parser	<code>moo/parser.py</code>	Command → verb / direct-object / preposition / indirect-object resolution
Verbs	<code>moo/verbs.py</code>	Verb compilation, preprocessing, caching, dispatch
Namespace	<code>moo/verb_namespace.py</code>	Builds the local scope every verb runs in
Matching	<code>moo/match_utils.py</code>	<code>pmatch</code> / <code>bmatch</code> natural-language object matching
Tasks	<code>moo/tasks.py</code>	Time-limited execution, <code>suspend</code> / <code>fork</code> , scheduling
Effects	<code>moo/effects.py</code>	Tick-driven, persisted timed-effect scheduling
Permissions	<code>moo/permissions.py</code>	Wizard/programmer/owner hierarchy, quotas
Builtins	<code>moo/builtins.py</code>	The function library injected into every verb's namespace
Strings	<code>moo/string_utils.py</code>	Gender-aware emit/pronoun substitution

World content lives in `moo verbs/<object-number>/<verb-name>.py` — 300+ verb files covering player commands and the staff/builder toolkit.

Scope. This manual documents MegaMOO the engine and its generic tooling. The repository also contains a specific game world built on the engine, with its own systems (character progression, combat, equipment, and so on). That game's design and content are

documented separately — this manual stays engine-only so the engine's capabilities are clear on their own.

What a verb looks like

`moo verbs/17/jump.py` — the `jump` command available in every in-character room:

```
"""
Jump across a gap or obstacle.

Usage: jump <exit>
"""

if not args:
    pobj.msg("Jump what?")
    return

# RT check
if (getattr(pobj, 'rt', None) or 0) > 0:
    pobj.msg("You must wait.")
    return

pos = getattr(pobj, 'position', 0) or 0
if pos:
    pobj.msg("You can't do that in your current position.")
    return

# Match exit in room contents
exit = pmatch(dobj, pobj, list(pobj.location.contents))
if not exit or not getattr(exit, 'is_exit', False):
    pobj.msg("Jump what?")
    return

if getattr(exit, 'jumpable', False):
    call_verb(exit, 'invoke')
elif getattr(exit, 'climbable', False):
    pobj.msg("You have to climb that!")
else:
    pobj.msg("You can't jump that!")
```

No imports, no boilerplate. The verb namespace arrives pre-loaded with the acting player (`pobj`), the parsed command text (`args`, `dobj`, `iobj` — all strings), object matching (`pmatch`, which turns that text into objects), cross-object calls (`call_verb`), and the rest of the builtin library. Objects are matched the way a player thinks — `jump gap`, `jump 2nd rope` — and the verb reads top to bottom like the action it performs. [Writing Verbs](#) explains every name in that namespace.

Design philosophy

MegaMOO is built with a deliberate set of trade-offs: it stays faithful to the MOO paradigm, keeps the engine small and dependency-free, and optimizes for the way text worlds are actually built and run. Each design choice below is stated with what it buys and what it costs.

1. **Faithful to the MOO paradigm.** The model is the product, not something approximated on top of a general-purpose framework. MegaMOO implements **prototypal, single-parent inheritance** over numbered objects directly; **in-game commands live on rooms** (the OOC/IC room parents) and dispatch to a `<verb>_` hook on matched objects inside verbs for per-object behavior; a real **natural-language parser** resolves articles, ordinals (`get 2 sword`), adjectives, possessives, and prepositions the way a player actually types. Nothing is bent to fit a class-per-table ORM. *Cost:* you don't inherit a framework's web/admin scaffolding — you build exactly what you want instead.
2. **Python is the language — hot-coded live, in-game.** Rather than implement a MOO-code interpreter, MegaMOO runs verbs as real Python with a thin preprocessor and a curated builtin library — the full standard library, native lists/dicts in properties, no DSL to learn. Verbs are plain files you can **write and edit from inside the running game** with the `@program` verb: on save it compiles and installs the verb into the live world (and writes the file to disk) in one step — **no reload, no restart, no build**. The change is live on the next invocation. The reverse direction is hot too: a background watcher picks up verb files edited on disk within a couple of seconds. Because the language is Python, the barrier to contributing is low and the pool of potential authors is everyone who already knows Python.
3. **Run many worlds at once.** A database is a **startup argument** (`megamoo.py <db> [port]`), so a development, staging, or experimental world runs **side by side with the live game** — each its own process and port — with no per-world install ceremony. Spinning up a scratch copy to test verbs and then promoting the work is trivial.
4. **Right-sized, not over-engineered.** Single-process, single-writer verb execution plus an embedded database is the correct fit for a text world's real load — commands are tiny and serialized, which sidesteps an entire class of data races for free. *Honest trade-off:* MegaMOO is not made to build a 10,000-player, multi-server MMO. MegaMOO is a modern Python and AI-enabled codebase engineered for building text-based multiplayer virtual worlds. Classic MOOs ran worlds of tens of thousands of rooms with hundreds of concurrent players on 1990s hardware; modern single-process hardware has magnitudes more headroom than any text world needs.
5. **Persistence without ceremony.** Objects live in SQLite (WAL mode for crash recovery) behind a lazy LRU cache, written through automatically when properties change. The hot working set stays in memory like a classic in-RAM MOO, but with durable, crash-safe persistence and **no**

code reload or stop-the-world checkpoint freeze. The world survives restarts without an explicit save step.

6. **Zero required dependencies.** The core engine is ~1 MB of pure Python on the standard library alone — no Django, no Twisted, no install tree to secure or upgrade. It is trivial to deploy, easy to read end to end, and (now that `asyncio` is in the stdlib) gives up little that a heavyweight framework would provide for a server of this kind.
7. **Accessibility is a feature of the engine, not a bolt-on.** MegaMOO is developed by a C1–C2 quadriplegic programmer using a head-pointer and typing on an on-screen keyboard at about 30 words per minute. A per-player `screenreader` mode strips all ANSI color and decoration; ANSI-aware wrapping measures visible width so escape sequences never break layout; server-side wrapping can be disabled entirely (`WRAP_WIDTH = 0`) for clients that reflow text themselves. Text worlds were the original accessible online games, and MegaMOO treats keeping them that way as part of its job.

Versions and conventions used in this manual

- **Engine version:** `0.10.0-beta20` , the single source of truth being `SERVER_VERSION` in `moo/globals.py` (printed by the startup banner and `megamoo.py --version`). The engine has not yet been load- or play-tested, so it is pre-release by design.
- **Object references** are written `#N` (e.g. `#3` is the staff base character). The core object hierarchy is summarized in [Architecture](#).
- **Source references** are written `module.py:line` or `moo verbs/<obj>/<verb>.py` and point at the current tree.
- **Permission levels** are written `gm1 ... gm5` ; see the [permission ladder](#).

License

MIT — see LICENSE.

01 — Architecture

This document describes how MegaMOO is put together: the object model that the whole world is built from, the path a single command travels from the socket to a running verb and back, and how state is persisted. Read it first — the rest of the manual assumes this vocabulary.

- [The object model](#)
 - [The core object hierarchy](#)
 - [The request lifecycle](#)
 - [The task system](#)
 - [Persistence](#)
 - [Networking and protocols](#)
 - [Concurrency model](#)
-

The object model

Everything in the world — players, rooms, exits, items, even the system object that holds global configuration — is a `M000bject` (`moo/objects.py`). Objects are identified by a stable integer (`objnum`, written `#N`) and arranged in a **single-parent inheritance tree**: each object has exactly one parent and inherits that parent’s properties and verbs, LambdaMOO-style.

Native attributes vs. the property system

A `M000bject` distinguishes two kinds of state:

- **Native attributes** are stored directly on the Python instance. The set is fixed (`_NATIVE_ATTRS` in `moo/objects.py`): `objnum`, `children`, `noun`, `aliases`, `owner`, `flags`, `properties`, `verbs`, `created`, `last_move`, `tags`, plus the relationship attributes `parent`, `location`, and `contents` (which have descriptor setters that keep both sides of the relationship in sync).
- **MOO properties** are everything else — `hp`, `description`, `gender`, any property a builder adds with `@adprop` or a verb adds with `add_property()`. These are stored in the object’s `properties` dict as `PropertyInfo` records (`moo/objects.py`), each carrying a `value`, an `owner`, and a `perms` string.

The split is invisible at the call site. Attribute access is routed through `M000bject.__getattr__` / `__setattr__`:

- Reading `obj.hp` checks local properties, then walks inheritance (via the cache below), then falls back to a verb of that name (returning a callable), and finally returns a falsy sentinel (`_null_attr`) if nothing matches — so unset properties read as falsy rather than raising.

- Writing `obj.hp = 5` updates the existing property in place or creates a new local property (default perms `rc`), and triggers a write-through to the database when one is attached.

This is why verbs read so cleanly: `pobj.location.contents`, `obj.hp -= damage`, and `getattr(exit, 'jumpable', False)` are all just attribute access, but they transparently respect inheritance, permissions, and persistence.

Property permissions

Each property's `perms` string mixes three flags:

Flag	Meaning
<code>r</code>	Readable by anyone (otherwise only the owner or a wizard)
<code>w</code>	Writable by anyone (otherwise only the owner or a wizard)
<code>c</code>	“Change/inherit” — children inherit a copy of this property

Permission enforcement happens in the verb namespace's safe `getattr / setattr` wrappers (`moov/verb_namespace.py`), not on the raw object — so engine-internal code can do what it needs, while verb code written by a non-wizard programmer is held to the property's declared permissions. See [Writing Verbs](#).

Inheritance, resolved cache, and invalidation

A naive inheritance lookup is $O(\text{depth})$: to read a property you might walk every ancestor. MegaMOO instead builds a **flattened resolution cache** per object — `_resolved_properties` and `_resolved_verbs` — mapping each inherited name to `(definition, defining_objnum)`. Lookups then become a single dict hit regardless of tree depth.

The cache is built lazily on first use (`_ensure_cache`) and **invalidated by cascading down the tree**: when an ancestor's properties or verbs change, the invalidation propagates to all descendants so the next lookup rebuilds. The practical payoff is that you can have deep class hierarchies (base object → item → container → furniture → a specific chair) without paying for the depth on every property read.

Verb resolution (`M000bject.find_verb`) follows the same shape: check local verbs first (so an object can override an inherited verb, and so the minimum abbreviation logic works locally), then consult the resolved-verb cache. Hidden verbs are skipped during player-initiated lookup.

Flags and tags

- **Flags** (`ObjectFlags`) are the classic MOO bits: `PLAYER`, `PROGRAMMER`, `WIZARD`, and so on. Some are kept in sync with the `auth` property — adding `gm3` sets `PROGRAMMER`, `gm4` sets `WIZARD` (see [Operations](#)).

- **Tags** are a free-form `category/tag` classification set (managed with `@adtag` / `@rmtag`) used for zones, game-system grouping, and queries.
-

The core object hierarchy

A handful of low-numbered objects form the backbone of any MegaMOO world, and the shipped database defines them out of the box: every object through **#54**, plus **#90/#91** (the consumable bases) and **#100**, the owner/wizard account and first character in the game. The table below lists the load-bearing ones; they are the conventional layout new content builds against:

#	Object	Parent	Role
#0	System object	—	Holds globals; <code>\$name</code> resolves to <code>#0.name</code>
#1	Root	#0	Ancestor of everything
#2	PlayerObjectDB	#1	Pool of player-account objects
#3	Base_Character	#10	Base for all characters; staff/builder verbs live here . Defines <code>is_char</code> .
#4	OCharacter	#3	Out-of-character (account) character
#5	ICharacter	#3	In-character (in-game) character — the puppetable avatar
#6	Nowhere	#1	Isolation container used during character generation
#7	arch	#22 GoExit	Character-generation entry exit in room #14
#8	Bag	#27	“The Body Bag”; holds the <code>moo_verb_path</code> config
#9	TrashBin	#1	Where recycled objects go
#10	BaseObject	#1	Base for all game objects
#11	object	#10	Generic object; defines the spatial put/get/look hooks
#12	item	#11	Carriable item
#14	(login room)	#16	OOO staging room where new connections arrive (<code>LOGIN_ROOM</code> in <code>moo/globals.py</code>)
#15	BaseRoom	#11	Base room; <code>gmove</code> / <code>match_exit</code>
#16	OCRoom	#15	OOO room; <code>go</code> / <code>look</code>
#17	ICRoom	#15	IC room; <code>go</code> / <code>look</code> plus round-time checks
#20	BaseExit	#11	Base exit; <code>invoke</code> / <code>gmove</code>
#21	DirectionalExit	#20	Virtual compass-direction exits

#	Object	Parent	Role
#22	GoExit	#20	Named, walkable passage exits
#23	ClosableGoExit	#22	Doors/gates that open, close, lock
#24	ClimbableExit	#22	Exits requiring <code>climb</code>
#25	JumpableExit	#22	Exits requiring <code>jump</code>
#26	BaseContainer	#12	Holds items; supports entry/exit
#30	BaseFurniture	#11	Sit/lie targets

Two structural conventions follow from this:

- **Where in-game verbs live.** Player commands are *not* defined on player objects. They live on the room parents — **#16 (OCRoom)** for out-of-character rooms and **#17 (ICRoom)** for in-character rooms — so every room of a type inherits the full command set (`look` , `go` , `get` , `inventory` , and so on). Character objects (#3/#4/#5) carry **only staff verbs**, inherited from #3. Per-object behavior is added not by putting a player verb on the object, but by the room verb calling a `<verb>_` hook on the matched object (e.g. `get` on the room calls `in_get` / `on_get` on the container). Room/player verbs use `pmatch()` and check `obj.existent` after matching; staff verbs on #3 use `bmatch()` . See [Writing Verbs](#).
- **The OOC / IC split** is realized as two character classes: you connect as an OCharacter (#4) in the login room (#14), then *puppet* into an ICharacter (#5) to play. The same player connection drives whichever object is currently puppeted. See [the request lifecycle](#) below and [Operations](#).

The request lifecycle

Here is the full path of a single typed command, e.g. `get 2nd sword from chest` .

1. Network read

A `PlayerConnection` (`moo/network.py`) owns one socket for its whole lifetime. Input is read line by line (capped at 8192 bytes), telnet protocol bytes are stripped/negotiated, and the resulting command string is handed to the parser.

2. Parsing

`CommandParser` (`moo/parser.py`) turns the line into a `ParseResult` :

1. Strip a leading `@` or `+` prefix if present (builder/utility commands).
2. Detect the `/expr` eval shortcut and route it to the `eval` verb.

- Find the verb by searching, in order, the **player**, the **player's location**, the **other objects in that room**, and finally the **player's inventory** — walking the inheritance chain at each step and stopping at the first match. In practice the player object only contributes staff verbs (from #3); the everyday commands — `look`, `go`, `get`, `inventory` — are found on the room (#16/#17), which is why a player standing in any room has the full command set even though nothing is defined on the player itself.
- Pull off MUSH-style switches (`look/brief` → verb `look`, switches `['brief']`).
- Split the remainder into a direct object, a preposition, and an indirect object (`get <dobj> from <iobj>`).

The result carries `verb`, `verb_obj` (where the verb was found), `dobj / dobjstr`, `prep`, `iobj / iobjstr`, `argstr`, `args`, and `switches`.

The parser also *attempts* object resolution, recording object numbers in

`ParseResult.dobj / .iobj`. **Those numbers are then discarded:** the verb type's parse (step 4 below) overwrites the namespace's `dobj / iobj` with plain strings, and the `ParseResult` is consulted only as a fallback if that parse fails. Verbs match their own objects — see [Matching objects](#).

3. Verb lookup

`M000object.find_verb` resolves the matched name against the object that owns it, honoring local overrides, the resolved-verb cache, minimum abbreviations, and the hidden flag. It returns the defining object number and the verb definition — the latter pointing at the `.py` source to execute.

4. Namespace setup and dispatch

The verb does not run inline on the event loop. `execute_command()` builds the namespace and hands the compiled code to a dedicated `ThreadPoolExecutor(max_workers=1)` — the single verb-execution worker — guarded by a 30-second `COMMAND_TIMEOUT`.

A top-level player command does **not** go through the task queue. Nothing in this path creates a `Task`. The queue and `VerbExecutor` (`moo/verbs.py`) serve the *other* entry points — work scheduled by `delay()` / `fork()` — which is also where `TaskLimits` applies. See [the task system](#).

`build_verb_namespace()` (`moo/verb_namespace.py`) constructs the scope the verb code will see: the injected Python builtins, the core context variables (`pobj`, `this`, `db`, `args`, ...), permission-checked `getattr` / `setattr`, the parsed command parts, messaging defaults, and the full MOO builtin library (`pmatch`, `call_verb`, `create`, `move`, `delay`, ..., plus the inherited `obj.msg` / `obj.msg_room` messaging verbs). This is enumerated in [Writing Verbs](#).

The parsed command parts in that namespace are produced by the verb's **verb type** — a pluggable class (default `MasterVerb`) that owns the per-verb `parse()` step, so coders can give a verb a custom argument syntax. See [Verb types: customizing the parser](#).

5. Preprocess, compile, execute

The verb's source is run through `preprocess_verb_code()` (`moo/verbs.py`): `#N` references become `db.get_object(N)`, `$name` references become system-object property access, and the body is wrapped in a function so that `return` works at top level. The result is compiled (and cached) and executed against the prepared namespace. The return value is captured as `namespace['result']`.

If the verb is a **generator** (it used `yield`), the engine starts an interactive session: each `yield` "prompt" sends a prompt to the player and the verb resumes with their next line of input. This is how multi-step flows that prompt the player (such as new-character setup) work mid-verb.

6. Output and persistence

Messaging (`obj.msg`, `obj.msg_room`, `broadcast`) runs the text through emit substitution and the per-connection color/wrap pipeline before writing to the socket. Any property writes the verb made are persisted through the object's write-through to SQLite. The task completes (or errors, or suspends), and the connection loops for the next line.

Connection, login, and puppeting

The lifecycle above assumes a logged-in player. Getting there:

1. A new connection sees a splash screen and the prompt `Enter your username or NEW to create a new account:`.
 2. **Login** (`moo/login.py`) authenticates against an existing account (passwords hashed with `bcrypt` when available, salted SHA-256 otherwise; inline `name password` on one line is supported, as is taking over an existing session). **NEW** runs an account-creation flow (`name` → `password` → `confirm`) that claims a free account object from the player pool (#2).
 3. The connected account object is flagged `PLAYER` and placed in the login room (#14) — the OOC staging area.
 4. From OOC the player **puppets** into an in-character `ICharacter` (#5). Puppeting swaps which object the connection drives and moves the IC character to its stored `last_location`. `exit` unpuppets back to OOC. On disconnect the engine unpuppets cleanly, saving `last_location` and clearing the `PLAYER` flag on the stored account so no "ghost" active flag remains.
-

The task system

Scheduled work runs inside a managed **task** (`moo/tasks.py`) — the queue behind `delay()` and `fork()`:

- **Resource limits** (`TaskLimits`): a tick budget (100,000), a wall-clock budget (5s), a maximum verb-call stack depth (50), and a maximum fork depth (10).
- `delay(seconds, code, context)` schedules a code string to run later without blocking. `fork(seconds, code, context)` does the same for independent follow-up work, with fork-depth limits to prevent fork bombs. `pause(seconds)` is the blocking alternative — it sleeps the shared worker, and is capped at 30s.
- **State machine:** `PENDING` → `RUNNING` → `COMPLETED`, with `SUSPENDED` (used by the scheduler for not-yet-due work) and `ERROR` / `ABORTED` states.

Two caveats worth stating plainly:

- **There is no `suspend()` builtin.** Verb code cannot park itself mid-execution and resume in place; `SUSPENDED` is an internal scheduler state, not an API.
- **`TaskLimits` does not bound top-level player commands**, which bypass the queue (see [step 4](#)). Their guard is the 30-second `COMMAND_TIMEOUT`, plus `MAX_VERB_DEPTH` inside `call_verb`. A verb that spins for 29 seconds will stall every other player, because all verb code shares one worker thread.

See [Writing Verbs](#) for the author-facing API.

Persistence

State lives in **SQLite** (`moo/database.py`), not in a flat memory image:

- **Write-through:** property changes on a database-attached object are persisted automatically; verbs do not call an explicit “save” in normal operation.
- **WAL mode** gives crash recovery and lets reads proceed during writes.
- **Lazy LRU object cache:** objects are materialized on demand and evicted under pressure, so a large world does not require holding every object in memory.
- **Checkpointing:** the server periodically writes checkpoint snapshots (interval and retention are configurable; see [Operations](#)) into a `<dbname>_checkpoints/` directory beside the database file — `mm.db` checkpoints into `mm_checkpoints/` — pruning old ones.
- **Database creation:** pointing the server at a non-existent database, or giving a template plus an output path, initializes a new database; see [Operations](#).

Verb source is stored twice, deliberately. The authoritative copy at runtime is the `code` column of the `verbs` table — that is what the engine compiles and executes. It is *mirrored* to disk under

`moo verbs/<objnum>/<verbname>.py` , resolved relative to the `moo_verb_path` config stored on #8, so the world's behavior is diff-able and version-controlled.

The two are kept in sync from both directions: `@program` compiles, installs into the live database, and writes the file in one step; and a background watcher (on by default, polling every 2s) re-loads any verb file whose mtime changes on disk, so edits from an external editor, `git pull` , or the MCP bridge go live without a command. `@reload` forces that same disk→database pull on demand. See [Hot coding](#).

Networking and protocols

`moo/network.py` serves Telnet and (optionally) WebSocket from the same asyncio event loop:

- **Protocol negotiation:** the server advertises MXP, GMCP, MSDP, and MSSP, and learns the client's terminal width via NAWS so wrapping matches the real window.
- **MXP clickable links:** text in backticks (``north``) is rendered as a clickable `<send>` link for MXP-capable clients and degrades to plain text otherwise.
- **Color:** `&`-prefixed codes (`&r` red, `&G` bright green, `&<245>` xterm-256 gray, `&<#FF0000>` hex RGB, `&n` reset) are translated to ANSI on output. See [Building Worlds](#).
- **ANSI-aware wrapping:** wrapping measures *visible* width, so escape sequences never count toward the line length or get split. `WRAP_WIDTH` (default 121) can be set to `0` to disable server-side wrapping entirely.
- **Screenreader mode:** the per-player `screenreader` command strips all color and decoration from that player's output stream, persistently.

Limits worth knowing: a cap on simultaneous connections, per-IP connection rate limiting, an idle timeout, an input-line size cap (8192 bytes), and a bounded number of login attempts before disconnect. Exact values live in `moo/globals.py` .

Concurrency model

MegaMOO is **asyncio for I/O, single-threaded for verb execution**. Network connections, timers, and the API are all coroutines on one event loop, but verb code runs on a single one-worker executor so that the object database is only ever mutated by one verb at a time. This sidesteps the entire class of data races that a multi-threaded object store would invite, and it matches the MOO mental model: the world advances one action at a time. Long-running work should be handed to `delay()` / `fork()` and the task scheduler rather than blocking that worker — `pause()` blocks it, and so does any slow verb.

The trade-off — a single CPU core for verb execution — is the right one for a text world, where the work per command is tiny and correctness of shared state matters far more than raw throughput.

Next: [Writing Verbs](#) — the namespace, the preprocessor, matching, and messaging, in author-facing detail.

02 — Writing Verbs

A verb is a Python file. When a player types a command that resolves to a verb, the engine builds a namespace, preprocesses the source, and executes it. This document is the reference for what you can rely on inside that file.

- [The shape of a verb](#)
- [The verb namespace](#)
- [The source preprocessor](#)
- [Matching objects](#)
- [Verb types: customizing the parser](#)
- [Messaging and emit substitution](#)
- [Calling other verbs](#)
- [Creating and changing objects](#)
- [Timing: pause, delay, and fork](#)
- [Interactive verbs with `yield`](#)
- [Permission-checked attribute access](#)
- [Conventions and idioms](#)

The shape of a verb

A verb file is bare Python — no function definition, no imports, no boilerplate. It runs top to bottom and may `return` at any point (the engine wraps the body in a function so top-level `return` is legal). Standard library `import` works when you need it, but most verbs need nothing beyond the injected namespace.

```
"""
Optional docstring. The first lines often serve as the in-game help text.

Usage: <verb> <args>
"""

if not args:
    pobj.msg("Do what?")
    return

# ... logic ...
```

The leading triple-quoted docstring is conventional: it documents usage and is where in-game help comes from.

The verb namespace

The namespace is assembled by `build_verb_namespace()` in `moo/verb_namespace.py`. Everything below is available as a bare name inside any verb.

Core context

Name	What it is
<code>pobj</code> / <code>player</code>	The acting player object (the two names are the same object).
<code>this</code>	The object the verb is defined on . Use it for the verb's own properties and helpers.
<code>caller</code>	The object that invoked this verb via <code>call_verb</code> (<code>None</code> for a top-level command).
<code>db</code>	The database singleton. <code>db.get_object(N)</code> fetches by number.
<code>location</code>	The player's current location (or <code>None</code>).
<code>verb</code>	The verb name as matched, e.g. <code>'look'</code> .
<code>args</code>	The argument string, stripped.
<code>argstr</code>	The raw, unstripped argument string as typed.

`pobj` vs `this` is the distinction that trips people up first: in `moo/verbs/17/jump.py`, `pobj` is the player jumping and `this` is the `ICRoom` the verb lives on. In a verb defined on a sword, `this` is the sword and `pobj` is whoever swung it.

Parsed command parts

Every one of these is a string (or a list of strings). None of them is a resolved object. `dobj` and `dobjstr` are the *same string*, as are `iobj` and `iobjstr` — the pair exists for familiarity, not because one is matched. Turning text into an object is always your verb's job; see [Matching objects](#).

Name	What it is
<code>dobj / dobjstr</code>	Direct-object text . Identical values.
<code>iobj / iobjstr</code>	Indirect-object text . Identical values.
<code>prep / preplist</code>	The preposition (<code>'in'</code> , <code>'on'</code> , <code>'from'</code> , ...) and its tokens.
<code>dobjlist / iobjlist</code>	The dobj/iobj strings split into tokens.
<code>dobj2 / prep2</code>	Secondary object text / preposition for multi-preposition commands.
<code>lhs / rhs</code>	Left/right of the preposition, for assignment-style verbs (<code>@desc x = y</code>).
<code>arglist</code>	<code>argstr</code> split on whitespace.
<code>switches</code>	Slash switches: <code>look/brief</code> → <code>['brief']</code> .
<code>match</code>	The preposition regex match object, or <code>None</code> .

For `put gem in box` you get `dobj == dobjstr == 'gem'` , `prep == 'in'` , and `iobj == iobjstr == 'box'` . To act on the gem you must match it yourself:

```
gem = pmatch(dobj, pobj, list(pobj.contents))
```

The command parser (`moo/parser.py`) *does* attempt its own object resolution and records object numbers on the `ParseResult` , but those numbers are discarded: the verb type's string parse overwrites them when the namespace is built. Its `ArgSpec` enum (`NONE / ANY / THIS`) is vestigial for the same reason — `VerbDef` has no `dobj_spec / iobj_spec` fields to declare, so the permissive defaults always apply.

Python builtins

A curated set is injected by name (`SAFE_PYTHON_BUILTINS` in `moo/verb_namespace.py`): `len` , `str` , `int` , `float` , `bool` , `list` , `dict` , `set` , `tuple` , `type` , `range` , `enumerate` , `zip` , `map` , `filter` , `reversed` , `sorted` , `sum` , `min` , `max` , `abs` , `round` , `all` , `any` , `isinstance` , `print` — plus the permission-checking `getattr / setattr` wrappers and a plain `hasattr` .

This is a convenience layer, not a security boundary. The namespace is a plain dict with no `__builtins__` key, and CPython injects the real builtins module into any such dict at `exec()` time. So `import` , `open` , `eval` , `exec` , and `__import__` all work inside verb code, and `import builtins; builtinsgetattr(...)` sidesteps the permission wrappers. The engine relies on this: `@adverb` itself does `import re` and `import os` to write verb stubs to

disk. Treat verb code as **trusted** code — the gm3 tier that can write verbs is effectively root on the server, which is why [the permission ladder](#) matters more than the builtin list.

The MOO builtin library

Injected from `moo/builtins.py`. Grouped by purpose:

Objects: `create(parent, owner)`, `recycle(obj)`, `valid(obj)`, `get_object(objnum)`, `move(obj, dest)`, `chparent(obj, new_parent)`, `max_object()`.

Properties: `add_property(obj, name, value=None, perms='rc')`, `delete_property(obj, name)`, `properties(obj)`.

Verbs: `add_verb(obj, names, perms='rx', ...)`, `delete_verb(obj, name)`, `verbs(obj)`.

Calling & scheduling: `call_verb(obj, verb_name, ..., **kwargs)`, `pause(seconds)`, `delay(seconds, code, context)`, `fork(seconds, code, context)` — see [Timing](#).

Messaging: `obj.msg(message, ...)` and `room.msg_room(message, ...)` are the verbs you call from verb code (defined on #1, so every object has them); `broadcast(message, ...)` reaches everyone connected. See [Messaging](#).

Matching: `pmatch`, `bmatch`, `match`, `match_all`, `omatch`, `smatch`, plus helpers `strip_articles`, `parse_ordinal`, `name_match`, `adj_match`, `prep_match`, `split_on_prep`.

Search: `search(query, ...)` / `find(query, ...)`.

Admin / lifecycle: `auth_level(obj)`, `sync_auth_flags(obj)`, `puppet(target)`, `force(player, command)`, `fire_hook(...)`, and the ticker functions `ticker_add(interval, verb_name, obj, idstring='')` / `ticker_remove(...)` / `ticker_remove_all(obj)` — hooks and the ticker are covered in [Engine Systems](#).

Helper modules: `su` (string utilities — emit/pronoun substitution), `ou` (object utilities — `make_room`, `make_object`, ...), `_effects` (the [effects manager](#) — note there is no bare `eu`; use `$eu`), and `globals` (the `moo.globals` module of shared constants — this shadows Python's `globals()` builtin).

The source preprocessor

Before compilation, verb source is rewritten by `preprocess_verb_code()` (`moo/verbs.py`). Two transformations matter:

Reference rewriting

A single master regex scans the source and rewrites object and system references **only in bare code** — strings and comments are matched first and left untouched.

You write	Becomes	Notes
<code>#42</code>	<code>db.get_object(42)</code>	An object literal.
<code>\$wearable</code>	<code>db.get_object(0).wearable</code>	A property on the system object (#0) — see named constants below.
<code>\$su</code>	<code>su</code>	The one special case: <code>su</code> is in <code>__PYTHON_CONSTANTS</code> , so it maps to the injected namespace name rather than a <code>#0</code> property.
<code>"text with #5"</code>	unchanged	Inside a string.
<code># a comment about #1</code>	unchanged	A real comment (<code>#</code> not followed by a digit).

The rewrite also reaches **inside f-string expressions**, so `f"You see {#5.name}"` correctly becomes `f"You see {db.get_object(5).name}"`. The distinction between “comment” and “object ref” is precisely “`#` followed by a digit is an object; otherwise it’s a comment,” which is why `#42` and `# note` coexist safely.

Named object constants

The `$` prefix gives well-known objects readable names. Every `$name` (except the special-cased `$su`) rewrites to `db.get_object(0).name` — a property read on the **system object #0**. Because those properties store object numbers, `$name` resolves to a well-known object without hardcoding its number:

```
# Instead of memorizing that the wearable base is #35:
glove = create(parent=$wearable)      # → create(parent=db.get_object(0).wearable)
if target.parent == $chest:           # readable, refactor-proof
    ...
$eu.trigger(pobj, 'poison', 5, 3)     # $eu → db.get_object(0).eu (the effects object,
                                     #53)
```

The constants defined on #0 in the shipped database are `$bed`, `$chair`, `$chest`, `$eu`, `$furniture`, `$globals`, `$hat`, `$item`, `$obj`, `$pants`, `$shirt`, `$shoes`, `$stable`, and `$wearable`. A `$name` with no matching property on #0 does not error — it reads as the falsy `_null_attr` sentinel, so a typo fails silently. Check the live list with `+props #0`.

This is the same indirection LambdaMOO’s `$foo` corewords provide: a layer of named aliases over raw object numbers, so verb code reads in terms of *roles* (“the wearable prototype”, “the effects utility”) rather than magic numbers, and a world can renumber its prototypes by re-pointing `#0`’s properties.

Adding one: set a property on #0 to the target object number — `@adprop #0.weapon = 200` (or from verb code, `db.get_object(0).weapon = 200`). After that, `$weapon` resolves to `#200`.

everywhere. The authoritative list of constants is simply whatever properties exist on `#0`; inspect it live with `+props #0`. Use short, lowercase names that name the role, and avoid Python keywords or builtin names.

Note: `su` and `ou` (string and object utilities) are injected directly into the namespace, so you write `su.wrap(...)` / `ou.make_room(...)` with no `$`. The effects manager is **not**: there is no bare `eu` name in the namespace. Reach it as `$eu` (which rewrites to `db.get_object(0).eu`, i.e. `#53`), or via the injected `_effects` module object.

Function wrapping

The body is wrapped so `return` works at top level:

```
# what you write
pobj.msg("Hello")
result = "done"

# what runs
def _verb():
    pobj.msg("Hello")
    result = "done"
result = _verb()
```

The wrapper's return value is captured as `namespace['result']` and becomes the value `call_verb` hands back to a caller. Compiled code is cached, so the preprocessing cost is paid once per verb version, not per invocation.

Matching objects

Turning `"2nd blue sword"` into the object the player means is the job of the matchers in `moo/match_utils.py`. Both major matchers resolve in the same order — possessives (`my sword` restricts to inventory), keywords (`me`, `here`), dbrefs (`#3`, `$utils`), then name matching against a candidate list.

Function	Returns	Use when
<code>pmatch(inp, pobj, candidates)</code>	The single best match, or <code>None</code> .	Player verbs. Supports <code>me</code> , <code>here</code> , <code>my <X></code> — but deliberately not <code>#N</code> or <code>\$name</code> , so players can't reach arbitrary objects.
<code>bmatch(inp, pobj, candidates, db=None)</code>	The single best match, or <code>None</code> .	Staff verbs. Same as <code>pmatch</code> plus <code>#N</code> / <code>\$name</code> dbref support.
<code>match(inp, candidates)</code>	The single best name match, or <code>None</code> (no keywords/refs).	Low-level matching against an explicit list.
<code>match_all(inp, candidates)</code>	A list of every name match.	When you genuinely want all matches.
<code>omatch(inp, pobj, db=None)</code>	Keyword/ref only (<code>me</code> , <code>here</code> , <code>#N</code> , <code>\$name</code>).	When you only want references, not names.

Note that `bmatch` returns one object, not a list — the difference from `pmatch` is dbref support, not arity. `match_all` is the one that returns a list.

Name matching understands how players actually talk:

- **Articles** are stripped (`the`, `a`, `an`, `some`).
- **Ordinals** select among duplicates: `2nd sword`, `third rope`, or a bare `3`.
- **Adjectives** filter in order: in `blue sword`, `sword` is the noun and `blue` must appear (as an in-order substring of the name, or in the object's `adjectives` list).
- **Aliases** and the atomic `noun` property are both checked, with multi-character tokens matched by prefix.

A standard player-verb matching block:

```
target = pmatch(dobj, pobj, list(pobj.location.contents))
if not target or not getattr(target, 'existent', False):
    pobj.msg("You don't see that here.")
    return
```

The `existent` check after matching is a project convention for **unhidden player verbs**: confirm the matched object still exists and is real before acting on it. Staff verbs on #3 use `bmatch` and generally skip that check.

Verb types: customizing the parser

The parsed command parts your verb reads (`dobj`, `prep`, `iobj`, `switches`, `lhs / rhs`, ...) don't appear by magic — they're produced by the verb's **verb type**, a small, swappable class that owns

the *parse* step and nothing else. Every verb has one, named by its `parent_type` : a dotted-path string stored in the `parent_type` column of the `verbs` table, defaulting to `moo.verb_types.MasterVerb` . The classes live in `moo/verb_types.py` .

Verb types are **engine Python**, not verb code. Changing one is a server-source change, not something you can do from `@program` — see [Adding a custom type](#) below. In day-to-day work you will use one of the two that ship, and most likely never think about this at all.

How a verb runs

Before your verb’s code executes, `_instantiate_verb_type()` (`moo/verb_namespace.py`) resolves the verb’s `parent_type` to a class via `resolve_verb_type()` , instantiates it, sets the runtime context on the instance (`pobj` , `this` , `location` , `db` , `cmdstring` , `raw` , `args` , and any injected switches), and calls:

```
inst.parse()
```

The attributes `parse()` leaves on the instance are then copied into [the namespace](#) your code sees. If the class can’t be instantiated or `parse()` raises, the engine logs a warning and falls back to simple string-split defaults, so the verb still runs with sensible `dobj` / `prep` / `iobj` values.

The lifecycle

Three of the four methods on `BaseVerb` are wired to the engine, and they fire around **every** execution of the verb — player commands, `call_verb` chains, and engine-invoked hooks like `go_` alike:

<code>at_pre_cmd()</code>	setup; return True to veto the command
<code>parse()</code>	split <code>self.args</code> into the standard slots
<the .py body>	your verb file — skipped if <code>at_pre_cmd</code> vetoed
<code>at_post_cmd()</code>	cleanup; runs even if the body raised or was vetoed

- **`at_pre_cmd()` runs before `parse()`** , which is what makes it the place for a check that should abort without paying for parsing — and equally why `dobj` / `prep` / `iobj` are *not* available to it yet. Return `True` to skip the body, the same “return True suppresses the default” convention the [hook system](#) uses.
- **`at_post_cmd()` always runs.** It reads the outcome off `self` : `self.result` (the body’s return value), `self.error` (the exception, or `None`), and `self.vetoed` .
- **Both fail safe.** An exception in `at_pre_cmd()` is logged and the command proceeds — one broken hook on a shared type must not silently swallow every command using it. An exception in `at_post_cmd()` is logged and swallowed, so cleanup can’t replace the failure it was reacting to.

func() is still not called by anything. Your verb's `.py` file *is* the body; there is no `func()` to override. It survives on `BaseVerb` only so that existing subclasses keep importing.

Example: one roundtime check instead of thirty

Every IC action verb opens with the same four lines — if the character is still in roundtime, refuse. A verb type turns that into a property of the verb rather than a paragraph each one repeats:

```
# in moo/verb_types.py (engine source)
@register_verb_type
class TimedVerb(MasterVerb):
    """An action a character in roundtime cannot take."""

    def at_pre_cmd(self):
        rt = getattr(self.pobj, 'rt', 0) or 0
        if rt > 0:
            self.pobj.msg(f"You must wait {rt} more seconds.")
            return True          # veto — the verb body never runs
```

Attach it and the check is inherited, with nothing left in the verb body:

```
add_verb(weapon, ['swing'], parent_type='moo.verb_types.TimedVerb')
```

Three things make this work where a plain early `return` in each verb would not:

- **The body never starts.** A veto is not an early return inside your code; the `.py` file is not executed at all, so there is no path through it that can forget the check.
- **It covers `call_verb` too.** A combat verb reached from another verb gets the same gate, which a copy-pasted guard at the top of player-facing verbs misses.
- **or 0 is load-bearing.** A missing property reads back as the `_null_attr` sentinel, which is falsy but is *not* `None` — `rt is None` is always `False`, and comparing the sentinel with `> 0` raises.

Because `at_pre_cmd()` runs before `parse()`, this pattern suits checks about the *actor* — roundtime, position, stun, permissions. A check that needs to know *what was targeted* has no `dobj` yet and belongs in the body.

The two built-in types

```
BaseVerb      Minimal – parse() is a no-op; args is the raw string.
└─ MasterVerb Standard dobj / prep / iobj / switches parsing (the default).
```

- **MasterVerb** does classic LambdaMOO-style parsing: it pulls MUSH switches off the verb name, finds the first preposition, and splits the rest into `dobj` / `prep` / `iobj` (plus `lhs` / `rhs`, a second preposition, and the list forms). This is what almost every verb wants.
- **BaseVerb** is “bring your own parser”: its `parse()` does nothing, so every parsed slot arrives empty — including `arglist`, which is `[]` rather than the split arguments. Only `args` / `argstr` are populated (they come from the namespace builder, not the verb type), so `args.split()` is on you. Use it for verbs whose syntax doesn’t fit the dobj/prep/iobj mould.

Remember that even under `MasterVerb` the parsed slots are **strings** — see [Parsed command parts](#). A verb type splits text; it never resolves objects. Matching is always your verb’s job.

Adding a custom type

There are two knobs, in increasing order of effort.

1. Swap the preposition regex (`rexp`). `MasterVerb.parse()` uses the class’s `rexp` in place of the global `PREP_REGEX` when one is set — the lightest way to change which words or separators split `dobj` from `iobj`.

2. Subclass and override `parse()`. Take full control of how `args` is split.

There is one rule that is easy to get wrong: **`parse()` must write its results into the standard slot names, because the engine harvests a fixed list.** `_parse_verb_inst_into_namespace()` copies exactly these attributes off the instance and nothing else:

```
dobj dobjlist prep preplist iobj iobjlist
dobj2 dobjlist2 prep2 lhs rhs arglist match switches
```

An attribute you invent — `self.field`, `self.value` — is **silently dropped**; your verb body will never see it. (`dobjstr` and `iobjstr` are derived from `dobj` / `iobj` at harvest time, so setting them directly does nothing either.)

So a `name: value` parser reuses the existing slots rather than inventing new ones:

```
# in moo/verb_types.py (engine source)
@register_verb_type
class KeyValueVerb(MasterVerb):
    """Parse 'name: value' pairs instead of dobj/prep/iobj."""
    def parse(self):
        key, sep, val = self.args.partition(':')
        self.dobj = key.strip()          # reads as dobj / dobjstr
        self.prep = ':' if sep else ''
        self.iobj = val.strip()         # reads as iobj / iobjstr
        self.lhs, self.rhs = self.dobj, self.iobj
        self.dobjlist = self.dobj.split()
        self.iobjlist = self.iobj.split()
        self.arglist = self.args.split()
        self.switches = getattr(self, '_injected_switches', []) or []
```

Anything you don't set keeps the empty default from `MasterVerb.__init__`, so a partial `parse()` degrades to blank slots rather than raising.

`@register_verb_type` is a **Python class decorator in engine source — not an in-game @-command**. It records the class in a registry keyed by its dotted path (`moo.verb_types.KeyValueVerb`) so `resolve_verb_type()` can find it by string. Registration is a fast path, not a requirement: `resolve_verb_type()` falls back to importing the module and doing a `getattr` for any path it doesn't already know.

Because this is engine code, the workflow is:

1. Add the class to `moo/verb_types.py` (or another module under `moo/` that the server process can import).
2. **Restart the server** (`@restart`). The auto-reload watcher only covers `moo verbs/`; engine modules are not hot-reloaded.
3. Attach it to a verb by setting that verb's `parent_type`.

Three ways to set `parent_type`:

How	Reaches
<code>@adverb <obj>.<name></code> with <code><perms></code> base	<code>BaseVerb</code> only — the in-game toolkit has no syntax for an arbitrary type.
<code>add_verb(obj, names,</code> <code>parent_type='moo.verb_types.KeyValueVerb')</code> from verb code	Any importable type. This is the normal route.
The JSON API <code>set_verb</code> request with a <code>parent_type</code> field	Any importable type. Note the MCP <code>set_verb</code> tool does not expose it — it passes only <code>objnum / verb / code</code> .

What the verb type does *not* control

`moo/verb_types.py` also defines `define_verb()`, `BaseVerb.matches()`, `to_dict()` / `from_dict()`, and the class attributes `key`, `aliases`, `min`, `perms`, and `help_text`. **None of these are used by the running engine** — nothing calls `define_verb()`, and verb identity lives entirely on the `VerbDef` (`moo/verbs.py`), which has its own `names`, `min_lengths`, `matches()`, `perms`, `hidden`, and `auth`. Setting them on a custom verb type has no effect; set them with `@adverb` / `@min` / `@verbauth` instead.

Messaging and emit substitution

There are three ways to put text in front of players, all of which run the text through the substitution engine (`moo/string_utils.py`) and then the recipient's color/wrap/screenreader pipeline:

- `pobj.msg(...)` — to one player.
- `room.msg_room(..., exclude=[pobj], sub=pobj, dob=target)` — to everyone in a room, optionally excluding actors.
- `broadcast(...)` — to everyone connected.

`msg` and `msg_room` are themselves verbs defined on #1 (`Root_Object`), so every object inherits them and verb code always reaches for `obj.msg(...)` rather than a free function. (Under the hood `msg` wraps a lower-level `notify()` primitive, but you write `obj.msg(...)`; because it's a verb, an object can even override it to filter or redirect its own messages — e.g. a deafened character.)

Substitution tokens

Pass the actors as `sub` (subject), `dob` (direct object), `iob` (indirect object), `uob` (noun source); the tokens then render per recipient:

Token	Renders
<code>%s</code> / <code>%S</code>	subject name / capitalized name
<code>%d</code> / <code>%D</code>	direct-object name / capitalized
<code>%i</code> / <code>%I</code>	indirect-object name / capitalized
<code>%u</code> / <code>%U</code>	noun (from <code>uob</code>) / capitalized
<code>%ps</code> <code>%po</code> <code>%pp</code> <code>%pa</code> <code>%pr</code>	gender pronouns of the subject: subjective / objective / possessive / possessive-adjective / reflexive

Both `%` and `$` prefixes are accepted for these tokens. Capitalize the token to capitalize the output (`%Ps` → “He”). Gender comes from the actor's `gender` property; the four supported

genders (male , female , neutral , ambiguous) map to pronoun sets in moo/globals.py , with ambiguous using singular they.

A canonical three-audience emit, from moo verbs/17/tap.py :

```
pobj.msg("You tap %d on the shoulder.", dob=tobj)
tobj.msg("%S taps you on the shoulder.", sub=pobj)
pobj.location.msg_room("%S taps %d on the shoulder.",
                       exclude=[pobj, tobj], sub=pobj, dob=tobj)
```

The actor sees “You tap Bob...”, Bob sees “Alice taps you...”, and the room sees “Alice taps Bob...” — one logical event, correctly conjugated for each viewer.

Pronoun helpers

For longer narrative strings, su.psub1(text, eobj=...) substitutes a single actor’s pronouns (%N / %CN name, %EPS / %EP0 / %EPP / %EPR and capitalized variants), and su.psub2(text, eobj=..., tobj=...) adds a target’s pronouns (%T / %CT , %OPS / %OP0 /...). Reach for these whenever a verb narrates an interaction between two characters.

Calling other verbs

call_verb(obj, verb_name, **kwargs) runs another verb and returns its result. Extra keyword arguments are injected into the called verb’s namespace, which is the standard way to pass structured data between verbs:

```
# A player verb on the room dispatches to the object's implementation:
call_verb(exit, 'invoke')

# Pass structured data into the called verb's namespace:
call_verb(this, 'on_use', actor=pobj, target=dobj)
```

Switch syntax works here too: call_verb(obj, 'look/brief') . Nested calls are bounded by the maximum stack depth (50) to prevent runaway recursion.

This dispatch pattern is pervasive: room-level verbs are thin and call object-specific _-suffixed implementations. The room’s sit verb matches the furniture and calls sit_ on it; get / put / look call the in_ / on_ / under_ / behind_ family on the target object. Keeping the player-facing verb a dispatcher lets each object type customize behavior without re-implementing parsing.

Creating and changing objects

Verbs can build the world at runtime:

```
sword = create(parent=db.get_object(35))    # child of #35 (BaseWearable)
sword.noun = "sword"
sword.description = "A plain iron sword."
add_property(sword, 'damage', 12)
move(sword, pobj)                          # into the player's inventory
```

- `create(parent, owner)` returns a new `M000object`.
- Property writes are just attribute assignment (`sword.damage = 12`), or `add_property` when you want explicit perms.
- `move(obj, dest)` relocates an object and fires the before/after move hooks.
- `recycle(obj)` deletes it (sending it to the trash).
- `chparent(obj, new_parent)` reparents.

The higher-level `ou` module (`ou.make_room`, `ou.make_object`, ...) wraps these for common world-building shapes, and the `@`-command toolkit in [Building Worlds](#) is built on exactly these primitives.

Timing: pause, delay, and fork

Three builtins schedule work. The distinction that matters is **blocking vs. not**: verb code runs on a single shared worker thread, so anything that blocks stops the whole world.

Builtin	Blocking?	Use for
<code>pause(seconds)</code>	Yes — sleeps the shared worker	Short beats in a sequence you accept freezing for. Capped at 30s.
<code>delay(seconds, code, context)</code>	No	The normal way to schedule later work.
<code>fork(seconds, code, context)</code>	No	Independent follow-up that runs as its own task.

`delay` and `fork` take the deferred work as a **code string** plus a `context` dict of the names it should see — they do not take a callable:

```
pobj.msg("You begin picking the lock...")
delay(3, 'pobj.msg("...click. The lock opens.")', {'player': pobj, 'pobj': pobj})
```

`context` must contain `'player'`; `delay / fork` raise `ValueError` without it. Deferred code is preprocessed exactly like verb source (so `#N` and `$name` work) and re-runs on the same single worker.

There is **no** `suspend()` in the verb namespace. The task system (`moo/tasks.py`) has a `SUSPENDED` state that `delay / fork` drive internally, but verb code cannot park itself mid-execution and resume where it left off. To pause a *conversation*, use `yield`; to pause *work*, use `delay`.

Task limits. `TaskLimits` (`moo/tasks.py`) defines a tick budget (100,000), a wall-clock budget (5s), a verb-call stack depth of 50, and a fork depth of 10. These bound *queued tasks* — the ones `delay / fork` create. A top-level player command does not go through the task queue at all (see [the request lifecycle](#)); its guard is the 30-second `COMMAND_TIMEOUT`, and the stack-depth limit is enforced separately by `call_verb` via `MAX_VERB_DEPTH`.

Round time (the `rt` property) is the gameplay-level cooldown layered on top of all this: most action verbs early-return if `pobj.rt > 0` and call a helper to set round time after acting. That is a game convention, distinct from the scheduler's limits.

Interactive verbs with `yield`

A verb that needs to ask the player something mid-execution becomes a generator by using `yield`. Each `yield "prompt"` sends the prompt and resumes the verb with the player's next line:

```
ans = yield "This will overwrite the existing inscription. Proceed? (y/n) "
ans = (ans or '').strip().lower()
if ans[:1] == 'y':
    this.inscription = new_text
    pobj.msg("You carve the new inscription.")
else:
    pobj.msg("You leave it as it was.")
```

The engine detects the generator return value and drives an interactive session automatically. Multi-step new-character setup and builder confirmations both use this. No callback plumbing required — the verb reads like a linear conversation.

Permission-checked attribute access

The `getattr` / `setattr` injected into the namespace are **not** Python's builtins — they are wrappers (`moo/verb_namespace.py`) that enforce MOO property permissions when the target is a `M000bject`:

- **Read** is allowed if the property is readable (`r`), or the player owns it, or the player is a wizard.
- **Write** is allowed if the player owns the property and it is writable (`w`), or the player is a wizard.

Non-`M000bject` targets pass straight through. `hasattr` is the standard builtin and does *not* enforce permissions, which is why the idiom for “read a property that may be unset” is `getattr(obj, 'name', default)` rather than a `hasattr` guard:

```
rt = getattr(pobj, 'rt', None) or 0          # unset → 0
if getattr(exit, 'jumpable', False):        # unset → False
    ...
```

Conventions and idioms

These are project conventions (see also the codebase memory and existing verbs), not engine requirements, but following them keeps new content consistent:

- **Define in-game commands on the room parents — #16 (OOC) and #17 (IC) — not on player/character objects.** Characters carry only staff verbs (from #3). To give an individual object custom behavior, have the room verb call a `<verb>_` hook on the matched object rather than adding a player verb to the object.
- **Player verbs use `pmatch` ; staff verbs (on #3) use `bmatch` .**
- **Send player output with `obj.msg(...)` / `obj.msg_room(...)` , not a raw `notify()` — `msg` is the inherited verb (on #1) every object exposes.**
- **Unhidden player verbs check `obj.existent` after matching** and treat a non-existent object as no match.
- **Pass actors explicitly to messaging:** `sub=pobj, dob=dobj, iob=iobj` so `%S / %d / %i` resolve. For `gmove` on exits, pass `sub=player, dob=this` (the exit) so `$d` substitutes to the exit.
- **Round-time gate first.** Action verbs check `pobj.rt / position / status` before doing work and set round time after.
- **Dispatch to `_`-suffixed implementations.** Keep the typed verb a thin matcher/dispatcher; put behavior on the target object's `verb_` method.
- **Hidden / internal verbs** are named with a trailing `_` (called programmatically) or a leading `_` (ticker callbacks, lifecycle hooks) and are marked hidden so players can't invoke them.

- **Colors:** prefer `<245>` for dim UI chrome; avoid `<240>` (too dark); `&n` resets. All of it disappears under screenreader mode, so never encode meaning in color alone.
-

Next: [Building Worlds](#) — the in-game `@`-command toolkit that drives all of the above.

03 — Building Worlds

World-building in MegaMOO happens **from inside the game**. A builder logs in, walks to where they want to create something, and types @-prefixed commands that create rooms, carve exits, make objects, and program behavior. These commands are themselves verbs (on #3, `Base_Character`), built on the primitives described in [Writing Verbs](#) — so the toolkit is just more MegaMOO, and you can extend it the same way.

This document is a reference to the in-game builder toolkit — around sixty commands — plus a worked example.

- [Permissions](#)
- [A worked example](#)
- [Rooms and exits](#)
- [Objects: create, name, describe](#)
- [Properties and tags](#)
- [Exit and action messages](#)
- [Programming verbs](#)
- [Inspection and debugging](#)
- [Colors and message tokens](#)
- [Movement and structure](#)
- [Command reference](#)

Permissions

Every builder command requires a minimum permission level. The ladder runs `gm1` (AssistantGM) → `gm2` (Builder) → `gm3` (Coder) → `gm4` (Admin) → `gm5` (God); see [Operations](#) for how levels are granted and how they map to the `PROGRAMMER` / `WIZARD` flags. As a rough guide:

- **gm2 (Builder):** dig rooms, open exits, make and describe objects, set messages. Everything needed to author content without writing code.
- **gm3 (Coder):** create objects from arbitrary parents, add/remove properties, program verbs, reparent, inspect internals, evaluate code.
- **gm4–gm5:** player administration, renumbering, number reservation, and server control.

Throughout this document each command notes its minimum level.

A worked example

Build a small two-room area with a connecting path and an item:

```
# 1. Dig an in-character room and teleport into it
@dig/tel ic = Mossy Clearing
@desc here = Sunlight filters through the canopy onto a carpet of soft moss.

# 2. Dig a second room (don't teleport yet)
@dig ic = Hollow Log

# ...note its object number from the output, say #142, then connect them:
@gopen path to #142          # named walkable exit, with an auto return exit
@vopen north to #142         # OR a virtual compass exit, if you prefer N/S/E/W

# 3. Make an item directly in the room (/drop), then dress it up
@make/drop #12 = lantern      # child of #12 (item); /drop places it here, not in
inventory
@desc lantern = A battered brass lantern, its glass smudged with soot.
@adjective lantern = battered brass

# 4. Inspect what you built
@examine here
+show lantern
```

Two stylistic choices appear here and recur throughout the toolkit:

- **Exits come in flavors** — directional vs. named, virtual vs. object-backed, plain vs. closable/climbable/jumpable. Pick the lightest one that fits.
- **Object identity is parent = name** — `@make <parent> = <name>` mirrors the `@dig <roomtype> = <roomname>` shape. The parent determines inherited behavior.

Rooms and exits

Rooms

Command	Level	What it does
<code>@dig[/types][/tel]</code> <code><roomtype> [= <name>]</code>	gm2	Create a room of <code>roomtype</code> (<code>/types</code> lists available types). <code>/tel</code> teleports you into it. Name defaults to the parent's noun.

`roomtype` selects the parent class — typically `ic` (#17 ICRoom) or `ooc` (#16 OCRoom).

Exits

MegaMOO distinguishes several exit kinds so movement reads naturally and so the engine can model doors, climbing, and jumping:

Command	Level	Creates	Use for
<code>@open <dir> [to <dest>]</code>	gm2	An object-backed directional exit (#21)	A compass exit you'll customize per-exit.
<code>@vopen <dir> to <dest></code>	gm2	A virtual directional exit stored in the room's <code>dexits</code>	Standard N/S/E/W links; lightest weight.
<code>@gopen[/noret] <name> [to <dest>]</code>	gm2	A named walkable passage (#22 GoExit)	"path", "archway", "trail".
<code>@dopen[/noret] <name> to <dest></code>	gm2	A closable door (#23) — starts closed	"gate", "door", "hatch".
<code>@copen[/noret] <name> [to <dest>]</code>	gm2	A climbable exit (#24)	"slope", "ladder", "wall".
<code>@jopen[/noret] <name> [to <dest>]</code>	gm2	A jumpable exit (#25)	"gap", "chasm", "ledge".
<code>@rmexit <exit></code>	gm2	—	Removes an exit (and its matching return exit).
<code>@virtualize <exit></code>	gm2	—	Converts an object-backed directional exit into a virtual one and recycles the original.

By default the `@*open` commands also create a **return exit** in the destination room; pass `/noret` to suppress that (one-way exits). Valid directions are

`north south east west ne nw se sw u d o in` — see the full [compass table and reverse mappings](#) (note `out / in` have no automatic reverse).

Virtual vs. object-backed exits. A virtual exit is just data in the room's `dexits` property — cheap, perfect for ordinary compass links. An object-backed exit is a real object you can give custom verbs, messages, locks, and keys. Start virtual; `@open / @gopen` when you need behavior.

Objects: create, name, describe

Command	Level	What it does
<code>@make[/drop] <parent> = <name> [with <owner>]</code>	gm3	Create an object as a child of <code>parent</code> (by <code>#num</code> , <code>\$const</code> , or name). <code>/drop</code> puts it in the room instead of your inventory.
<code>@delete <object></code>	gm2	Permanently recycle an object (confirms first; core objects #0–#9 are protected).
<code>@name <object> = <name></code>	gm2	Set the noun and auto-detect the article (capitalized → proper noun, no article).
<code>@desc <object> = <text></code>	gm2	Set the description (what <code>look</code> shows). <code>@desc x =</code> clears it.
<code>@adjective <object> = <a1> [a2] [a3]</code>	gm2	Set up to three adjectives (matched by <code>pmatch</code>). Clear with <code>@adjective x =</code> .
<code>@article <object> = <a/an/the/some></code>	gm2	Override the article.
<code>@trailer <object> = <text></code>	gm2	Set trailing text in the title (“a sword <i>with a golden hilt</i> ”).
<code>@title <object></code>	gm2	Regenerate the display title from noun + modifiers.

An object’s display name is composed from `article + adjectives + noun + trailer`, which is why these are separate commands — each piece participates in matching and in how the name renders in different grammatical positions.

Properties and tags

Command	Level	What it does
<code>@adprop <object>.<prop> [= <value>]</code>	gm3	Add a new property. The value is evaluated as a Python expression first (<code>[]</code> , <code>0</code> , <code>{}</code>); if that fails it's stored as a string.
<code>@set <object>.<prop> [= <value>] (@val)</code>	gm3	Set an existing property (local or inherited), or read it if <code>= <value></code> is omitted. Same literal-then-string evaluation as <code>@adprop</code> . Records the previous value.
<code>@unset <object>.<prop></code>	gm3	Revert the last <code>@set</code> on that property.
<code>@rmprop <object>.<prop> (@remprop)</code>	gm3	Remove a locally defined property (confirms).
<code>@clear[/all] <object>[.<prop\ verb>]</code>	gm3	Clear one local property/verb, or with <code>/all</code> every local property.
<code>@adtag <object> = <category>[/<tag>]</code>	gm3	Add a classification tag (e.g. <code>zone/haven</code>).
<code>@rmtag <object> = <category>[/<tag>]</code>	gm3	Remove a tag, or an entire category if no <code>/tag</code> given.

`@adprop here.spawns = []` then editing it from a verb is the typical way to add custom per-object state that game logic reads later.

Exit and action messages

Exits (and other usable objects) carry message pairs: one shown to the actor, one (`o` -prefixed) shown to onlookers, with emit tokens like `%S` .

Command	Level	Shown to	When
@success <obj> = <msg>	gm2	actor	On successful use (“You head north.”)
@osuccess <obj> = <msg>	gm2	others in the source room	“%S heads north.”
@failure <obj> = <msg>	gm2	actor	On failed use (“The door is locked.”)
@ofailure <obj> = <msg>	gm2	others	“%S struggles with the door.”
@drop <obj> = <msg>	gm2	actor on arrival	“You arrive at the clearing.”
@odrop <obj> = <msg>	gm2	others in the destination room	“%S arrives.”

Clearing any of them is @<cmd> <obj> = with an empty right-hand side. See [message tokens](#) for the substitution vocabulary.

Programming verbs

This is where building shades into coding. Verbs can be created and edited entirely in-game *or* on disk, and the two stay in sync.

Command	Level	What it does
<code>@adverb[/hidden] <obj>.<name[(min)] [,alias...]> [with <perms> [base] [min=N] [auth=N]]</code>	gm3	Add a verb. Names can carry a minimum-abbreviation length in parens; commas add aliases. <code>perms</code> defaults to <code>rx</code> ; <code>auth=N</code> sets the minimum gm level to invoke; <code>/hidden</code> makes it non-invokable by players (for hooks).
<code>@program <obj>.<verb></code>	gm3	Open the in-MOO line editor for a verb. On save it compiles, installs the verb into the live server immediately, and writes the source to disk — see Hot coding below.
<code>@reload <obj>.<verb> / @reload <obj> / @reload all</code>	gm3	Force a pull of verb source from disk into the running server. Rarely needed — the auto-reload watcher normally does this for you (see below). On first use for an object it creates the per-object directory and exports existing verbs; <code>@reload all</code> scans every directory and loads/creates/updates.
<code>@rmverb <obj>.<verb></code>	gm3	Remove a locally defined verb (confirms).
<code>@adalias <obj>.<verb> = <alias></code>	gm3	Add another name to an existing verb. Refuses a name already used by another verb on that object.
<code>@rmalias <obj>.<verb> = <alias></code>	gm3	Drop one alias from a verb, along with its <code>@min</code> setting. Refuses to remove the primary (first) name — use <code>@rmverb</code> for that.
<code>@min <obj>.<verb> = <N></code>	gm3	Set how many characters a player must type to match the verb.
<code>@hideverb <obj>.<verb></code>	gm3	Hide a verb (e.g. an internal hook like <code>at_post_move</code>).
<code>@unhideverb <obj>.<verb></code>	gm3	Un-hide a hidden verb, making it player-invokable again.
<code>@verbauth <obj>.<verb> [= <level>]</code>	gm3	View or set the minimum auth level required to invoke a verb.

Hot coding, no reloads

Editing a verb with `@program` is **hot** — there is no separate reload or restart step. When you finish the editor (`.` on a line of its own), the engine compiles the code, and on success **installs it into the live database right then**: an existing verb’s bytecode is recompiled in place, a new verb is added to the object. The change is active on the *next* invocation of the verb. `@program` *also* writes the source to `moo verbs/<objnum>/<verbname>.py`, so the live world and the on-disk copy never drift. If the code has a syntax error, nothing is installed and nothing is saved — the live verb keeps running.

Disk edits are hot too: the auto-reload watcher

The disk direction is also automatic. The server runs a background watcher (`dev.autoreload_verbs`, **on by default**, polling every `dev.autoreload_interval` = 2 seconds) that compares file mtimes under `moo verbs/` and re-loads any verb whose file changed. Edit `moo verbs/17/wave.py` in a normal editor, save, and the change is live a couple of seconds later — no command, no restart.

`@reload` is the manual version of that same disk→database pull. You need it only when you don’t want to wait for the poll, or when the watcher is disabled in config. You do *not* run it after

`@program` .

The two editing loops

In-game (hot):

1. `@program #17.wave` — type the code, end with `.` .
2. Done. It’s compiled, live, and written to disk in one step.

On disk (also hot):

1. `@adverb #17.wave` to declare the verb if it doesn’t exist yet.
2. Edit `moo verbs/17/wave.py` in a normal editor — or via the [MCP integration](#), which exposes `get_verb` / `set_verb` to an AI assistant.
3. Save. The watcher picks it up within ~2 seconds. (`@reload #17.wave` if you want it *now*.)

Because verb source is mirrored to plain files, the whole world’s behavior is diff-able and version-controlled — while remaining editable live from inside the game, with no build or restart in either loop.

What is *not* hot: engine Python under `moo/` — builtins, verb types, the parser, the namespace builder. Those changes need `@restart` .

Inspection and debugging

Command	Level	What it shows
@examine <object>	gm3	Parent, owner, location, flags, children, contents, all local properties (values truncated), and local verbs with line counts.
+show <object>	gm1	Noun/name/id, the full parent chain and location chain, owner, flags, children, contents.
+props[/all] <object>	gm3	Local property names (or, with /all , the whole inheritance chain grouped by object).
+verbs[/all] <object>	gm3	Local verb names (or the full chain); * marks the minimum-abbreviation point.
+decompile <object>.<verb>	gm3	Print a verb's source (requires r permission on the verb).
@list [<start>] [to <end>]	gm3	Objects in a number range, with parent and name.
eval <expr> / / <expr>	gm3	Evaluate arbitrary Python in the verb context. / len(db.objects()) , / pobj.objnum .
+pron	gm2	Reference table of pronoun substitution tokens.
@color	gm1	Reference table of color codes.

me and here are valid object references everywhere (+show here , @examine me), as are #N and \$name .

Colors and message tokens

Color codes

Output codes are %-prefixed and translated to ANSI:

Code	Effect
<code>&r &g &b &y &m &c &w &k</code>	red / green / blue / yellow / magenta / cyan / white / black
<code>&R &G &B ...</code> (uppercase)	bold/bright variants
<code>&<245></code>	xterm-256 color (245 = dim gray, good for UI chrome)
<code>&<#FF0000></code>	hex RGB
<code>&n</code>	reset

Conventions: `&<245>` for dim chrome, avoid `&<240>` (too dark). **All color is stripped under screenreader mode**, so never rely on color alone to convey meaning. `@color` prints the live reference.

Emit tokens

The same tokens from [Writing Verbs](#) apply in builder messages (`@osuccess`, etc.). Both `%` and `$` prefixes work:

Token	Renders
<code>%S / %s</code>	subject's capitalized / plain name
<code>%D / %d</code>	direct object
<code>%ps %po %pp %pa %pr</code>	subject's gender pronouns

`+pron` prints the live reference.

Movement and structure

Command	Level	What it does
<code>@tel <dest> / @telq <dest></code>	gm1	Teleport to <code>#N</code> , <code>home</code> , <code>mark</code> , or a character's location. <code>@telq</code> is silent (no departure/arrival messages). Won't cross the IC/OOC boundary.
<code>@move <obj> to <dest> [with <msg>]</code>	gm2	Move an object to a room/container, with an optional arrival message.
<code>@parent <obj> = <new_parent> (or a #start to #end range)</code>	gm3	Reparent one object or a range.

Game-specific staff commands. The `moo verbs/3/` directory also contains commands that exist only to drive a particular game's systems (character progression, body/equipment

models, creature spawning, and the like). Those are part of the game built on the engine, not the engine itself, and are documented with the game rather than here.

Command reference

The full toolkit grouped by purpose. Minimum level in parentheses.

Rooms & exits: @dig (2), @open (2), @vopen (2), @gopen (2), @dopen (2), @copen (2), @jopen (2), @rmexit (2), @virtualize (2)

Objects: @make (3), @delete (2), @name (2), @desc (2), @adjective (2), @article (2), @trailer (2), @title (2)

Properties & tags: @adprop (3), @set / @val (3), @unset (3), @rmprop / @remprop (3), @clear (3), @adtag (3), @rmtag (3)

Messages: @success (2), @osuccess (2), @failure (2), @ofailure (2), @drop (2), @odrop (2)

Verb programming: @adverb (3), @program / @code / @prog (3), @reload (3), @rmverb (3), @adalias (3), @rmalias (3), @min (3), @hideverb (3), @unhideverb (3), @verbauth (3)

Inspection: @examine (3), @list (3), +show (1), +props (3), +verbs (3), +decompile (3), eval / / (3), +pron (2), @color (1)

Structure & movement: @parent (3), @move (2), @tel / @telq (1)

Player admin: @auth (5), @delplayer (4)

Database & server: @renumber (5), @reserve (5), @freeon / @freeoff (3), @shutdown (5), @restart (5)

The player-admin and server commands are covered in [Operations](#).

Next: [Operations](#) — running the server, configuration, the JSON API, and the MCP integration.

04 — Operations

This document covers running and administering a MegaMOO server: starting it, configuring it, the permission model, persistence and backups, and the JSON API / MCP integration that lets external tools (including AI assistants) drive the live world.

- [Requirements](#)
- [Starting the server](#)
- [Command-line reference](#)
- [Creating a database](#)
- [Configuration](#)
- [The permission ladder](#)
- [Player administration](#)
- [Persistence and backups](#)
- [Logging](#)
- [Shutdown and restart](#)
- [The JSON API](#)
- [The MCP integration](#)

Requirements

- **Python 3.10+.** No `pip install` is needed for the core server — it runs on the standard library alone.
- **Optional:** `bcrypt` for stronger password hashing (a salted SHA-256 fallback is built in); `websockets` for browser clients; `mcp` for the AI integration bridge.

Starting the server

For everyday development use the launcher, which needs nothing but a database:

```
./mm mm.db
```

It enables the API, passes the shared token from `~/.megamoo/token` (generated on first run), turns on verb auto-reload, and names no ports — so a second database is started exactly the same way, with no arguments to keep distinct:

```
./mm ~/megamoo/mm.db
```

With no argument at all it picks the only `.db` in the repo root, and says so rather than guessing when there are several. Anything after the database is passed through to `megamoo.py` (`./mm mm.db --log-level DEBUG`).

The launcher is a thin wrapper over the real entry point, `megamoo.py` :

```
python3 megamoo.py mm.db
```

That loads the database `mm.db` and listens for Telnet on the **default port 6770** (`DEFAULT_PORT` in `moo/globals.py`) — or the next free port above it; see [Port selection and discovery](#). Connect with any MUD client or:

```
telnet localhost 6770
```

You'll see the splash screen and the prompt `Enter your username or NEW to create a new account: .`

Port note: the project README's quick-start uses `7777` as an *example* value, but the actual built-in default is `6770`. Pass `--port` (or a positional port) to pin one explicitly — which also turns off the search for a free port, so a conflict fails the boot instead of moving the game somewhere players aren't looking.

The argument parser supports both a LambdaMOO-style positional form and modern flags, and is smart about positional arguments — a second positional that is all digits is treated as a port, a hostname-like token as the bind host:

```
python3 megamoo.py mm.db                # db, default host/port
python3 megamoo.py mm.db 7777            # db on port 7777
python3 megamoo.py mm.db localhost 7777  # db, bind localhost, port 7777
python3 megamoo.py --input mm.db --port 8888 --host 127.0.0.1
```

On startup the server prints a banner with the version, loads the database, opens the listener(s), restores tickers, and starts its background tasks (checkpointing, the task queue, the ticker loop).

Status: the engine is at `0.10.0-beta20` and has **not yet been load- or play-tested**. Treat it as pre-release: suitable for development and single-developer worlds, not yet for an open multiplayer deployment. The version shown is `SERVER_VERSION` in `moo/globals.py`, the single source of truth for the build.

First login: the owner/wizard account (#100)

The shipped database includes **#100**, the default owner/wizard account — the first character in the game. It has full authority (`gm5` , plus the `WIZARD` and `PROGRAMMER` flags), so it can create players, build rooms, write verbs, and administer the server. `megamoo init` generates a password for this account and prints it once, when it creates the game:

```
Wizard login: Wizard / winter-timber-7382
Shown once, and unique to this world -- write it down.
```

Every world gets its own. There is no default password to look up, and none in the released package – a shipped hash is a shipped secret, and this one was recoverable from the wheel in under a second.

If you have lost it, there is no recovery path from outside the game; create a fresh world, or set the property directly against the database with the server stopped.

Two things to do on first login, before anything else:

1. **Change the password.** Use `password` (interactive) or `setpass <newpassword>` from the OOC lobby. **Do this immediately** in any environment reachable by others — it is the first item on the [security checklist](#).
2. **Make it yours — rename #100.** This account is meant to *become* the owner's own character, not stay a generic "Wizard." Rename it with `@rname #100 = <YourName>` , which moves the login name with it — you log in as the new name from then on, and `Wizard` stops working.

Use `@rname` , not `@name` , for an account. `@name` renames the object and leaves the login where it was, so you would end up called by your own name while still typing `Wizard` at the prompt. `@name` remains the right verb for everything that is not an account (see [naming objects](#)).

From here you grant other staff their tiers with `@auth` (see [the permission ladder](#)).

Keep at least one account at `gm5` . If you rename and re-secure #100 it remains your god account; only create additional `gm5` accounts deliberately.

Command-line reference

Flag	Default	Purpose
database (positional) / --input, -i	— (required)	Database file to load.
new_database (positional) / -- output, -o	—	Create a new database from the template (see below).
port (positional) / --port, -p	first free from 6770 up	TCP port for the Telnet listener. Naming one pins it: a conflict then fails the boot.
--host	0.0.0.0	Bind address.
--config, -c	—	Path to a ServerConfig JSON file (see Configuration).
--api	off	Enable the JSON API server (first free port from 7778 up).
--api-port	auto	Pin the API to this exact port; fails if it is in use.
--api-token	—	Shared secret API clients must present.
--web	off	Enable the browser client: serves the client and accepts WebSocket players (first free port from 8888 up).
--web-port	auto	Pin the browser client to this exact port; fails if it is in use.
--web-host	follows --host	Bind the browser client somewhere the game listener isn't. Set 127.0.0.1 when a reverse proxy fronts the client, so the plain HTTP port isn't reachable from outside the box.
--web-origins	same-origin only	Comma-separated origins allowed to open a WebSocket in addition to the one the client was served from (e.g. https:// play.example.com). Needed only when the client is served from somewhere other than this server. * accepts any origin.
--web-tls	off	Serve the browser client over HTTPS, using --tls-cert / -- tls-key . A flag on the web port rather than a second one, because the client picks ws:// or

Flag	Default	Purpose
		<code>wss://</code> from the page it was served over.
<code>--tls-port</code>	—	Serve an additional TLS listener on this exact port. The plain port keeps working — telnet cannot speak TLS — so this is a second door to the same world. Requires <code>--tls-cert</code> and <code>--tls-key</code> .
<code>--tls-cert</code>	—	PEM certificate for <code>--tls-port</code> , including any intermediates.
<code>--tls-key</code>	—	PEM private key for <code>--tls-port</code> .
<code>--log-level</code>	INFO	DEBUG, INFO, WARNING, or ERROR.
<code>--dev</code>	off	Development mode: pick the lone <code>.db</code> here, reuse the shared API token, publish a discovery file so tooling can find the world, and hot-reload verbs from disk. Turns on <code>--api</code> and <code>--web</code> as well.
<code>--version</code> , <code>-v</code>	—	Print the version and exit.
<code>-h</code> , <code>--help</code>	—	Full help.

Creating a database

Give the server a **template** plus an **output path** and it will build a fresh database from the template and exit, telling you how to launch it:

```
python3 megamoo.py template.db newworld.db
# ...creates newworld.db, then:
python3 megamoo.py newworld.db
```

Pointing the server at a database file that doesn't exist (with no output path) is an error — it won't silently create an empty world; you start from a template.

The shipped database. MegaMOO ships with a default database containing the core object hierarchy — every object through [#54](#), plus [#90](#) (BaseEdible) and [#91](#) (BaseDrinkable) — and [#100](#), the owner/wizard account and the first character in the game. That gives you the base

prototypes (rooms, exits, containers, furniture, wearables, consumables) and a usable login from the first launch; build your world on top of it.

Configuration

Two distinct files are easy to confuse:

- **`config.json` in the repo root is *not* the server config.** It configures an unrelated document/vector-search tool (embeddings model, chunk size). The server does not read it.
- **The server's `--config` expects a `ServerConfig` JSON** with `network`, `database`, `protocol`, `api`, and `dev` sections, matching the dataclasses in `moo/config.py`. Anything omitted falls back to the documented default. `ServerConfig` also carries top-level scalars — `server_name`, `motd`, `login_welcome`, `display_screen`, `max_command_length`, `enable_color`, `debug_mode`, `log_level`.

A representative config (all values shown are the defaults):

```

{
  "network": {
    "host": "0.0.0.0",
    "port": 6770,
    "auto_port": true,
    "port_scan_limit": 50,
    "max_connections": 100,
    "connection_timeout": 3600,
    "max_players": 0,
    "tls_port": 0,
    "websocket_enabled": false,
    "websocket_port": 8888
  },
  "database": {
    "path": "game.db",
    "checkpoint_interval": 3600,
    "max_checkpoints": 10,
    "transaction_log_enabled": true,
    "auto_save_interval": 300,
    "compress_checkpoints": true,
    "backup_on_start": true,
    "max_object_cache": 1000
  },
  "protocol": {
    "...": "MXP / GMCP / MSDP / MSSP toggles"
  },
  "api": {
    "enabled": false,
    "port": 7778,
    "auto_port": true,
    "port_scan_limit": 50,
    "info_path": "",
    "host": "127.0.0.1",
    "auth_token": "",
    "testbot_objnum": 0
  },
  "dev": {
    "autoreload_verbs": true,
    "autoreload_interval": 2.0
  }
}

```

Notes on the fields that matter most in practice:

- **network.max_players: 0** means unlimited; **max_connections** bounds raw sockets.
- **network.auto_port** lets a busy **port** move the listener up to the next free one instead of failing the boot, which is what makes a second database a one-word command. Set it **false** (or name a port on the command line) when the server must be reachable at a fixed address.

- `database.auto_save_interval` (5 min) is the in-memory-to-disk save; `checkpoint_interval` (1 hr) writes pruned recovery snapshots; the two are independent. `backup_on_start` takes a full copy before the server begins writing.
- `api.host` defaults to `127.0.0.1` — keep it that way unless you really mean to expose the API, and always set a strong `auth_token`. `testbot_objnum` pins the API's driver character (see [MCP](#)).
- `dev.autoreload_verbs` is `true` by default. A background watcher polls `moo verbs/` every `dev.autoreload_interval` seconds and hot-loads any verb file whose mtime changed, so external edits go live without `@reload` (see [the auto-reload watcher](#)). Set it to `false` if you want disk edits to land only when explicitly pulled.

The config validates on load: SSL requires a cert/key, an enabled API requires a valid port, etc.

The permission ladder

Authorization is a five-level ladder stored in each character's `auth` property and managed in-game. Two object flags are kept in sync automatically: granting `gm3` sets `PROGRAMMER`, granting `gm4` sets `WIZARD`.

Level	Name	Can, roughly
gm1	AssistantGM	Inspect (<code>+show</code>), teleport, basic helpers.
gm2	Builder	Full world-building: rooms, exits, objects, descriptions, messages.
gm3	Coder	Programming: create from any parent, add/remove properties and verbs, reparent, <code>eval</code> , inspect internals.
gm4	Admin	Player administration (<code>@delplayer</code>).
gm5	God	Server control (<code>@shutdown</code> / <code>@restart</code>), renumbering, number reservation, granting auth.

Grant and inspect with `@auth` (gm5 only):

```
@auth bob = add gm2
@auth bob = remove gm3
@auth bob = list
```

Per-verb authorization is independent of who *owns* a verb: `@verbauth` and the `auth=N` option on `@adverb` set the minimum level required to invoke a specific verb (see [Building Worlds](#)).

Player administration

Command	Level	What it does
<code>@auth <player> = add\ remove <level> / = list</code>	gm5	Manage a player's auth levels (syncs PROGRAMMER / WIZARD).
<code>@delplayer <object#></code>	gm4	Permanently delete an account: moves its IC characters and inventories to storage (#8), clears local properties, returns the account to the player pool (#2). Requires typing <code>YES</code> to confirm.

Persistence and backups

State lives in **SQLite** with WAL journaling. In normal operation you do not issue saves — property writes are persisted through automatically (see [Architecture](#)). For operations:

- **Auto-save** flushes memory to disk every `auto_save_interval` seconds (default 300).
- **Checkpoints** are written every `checkpoint_interval` seconds (default 3600) into a directory named after the database file and placed beside it — `mm.db` → `mm_checkpoints/`, holding `checkpoint_<timestamp>.sqlite` snapshots — with `max_checkpoints` retained and optional gzip.
- **backup_on_start** copies the database before the server begins writing.
- The WAL files (`*.db-wal`, `*.db-shm`) alongside the database are normal SQLite artifacts; don't delete them out from under a running server.

To take a manual backup, the safest path is to `@shutdown` (which saves and unpuppets everyone) and copy the `.db` file. A live file copy can be torn if the server is mid-write — acceptable for a scratch snapshot, not for an authoritative backup.

Logging

The server logs to `megamoo.log` in the working directory via a rotating file handler (10 MB per file, 5 backups) and also to the console. The level defaults to `INFO`; raise it with `--log-level DEBUG`

when diagnosing. Because the log is a plain file, it can be tailed even when the server is down — the MCP bridge exposes exactly that (`tail_log`).

Shutdown and restart

Both are gm5 verbs and both save the database, broadcast an optional message, and unpuppet every connected character before disconnecting:

```
@shutdown Server going down for maintenance.
@restart Applying a hotfix, back in a moment.
```

`@restart` re-executes the server process after a clean save, so it picks up changes to **engine Python under moo/** — builtins, verb types, the parser, the namespace builder. That is the one category that is not hot.

It re-runs the same command line, so every launch flag survives — `--dev` , `--web` , `--port` , `--log-level` , the database — and so does the working directory. Two flags are handled specially, and deliberately not the same way:

	Behaviour on restart	Opt out / in
<code>--api</code>	Forced on , however the server was launched, so tooling reconnects without anyone remembering a flag.	<code>@restart/noapi</code> (or the bare word <code>noapi</code>)
<code>--web</code>	Left as it was . Only ever added, never stripped.	<code>@restart/web</code> adds it

The asymmetry is the point: the API is one loopback socket, while the browser client is reachable by anything that can reach the host, and without `--web-origins` that is an exposure rather than a convenience. A restart must not switch it on for a server that never asked.

`--dev` implies both, so on a development server neither switch changes anything.

You almost never need it for game content: verbs written in-game with `@program` are live immediately, and verbs edited on disk are hot-loaded by the auto-reload watcher within a couple of seconds (or on demand with `@reload`). See [Hot coding](#).

The JSON API

When started with `--api` , the server runs a newline-delimited JSON-over-TCP API (`moo/api.py`), **off by default and localhost-only** by default. It is the control plane for external tooling — editors, scripts, and the MCP bridge. Enable it with a token:

```
python3 megamoo.py mm.db --api --api-token <token>
```

The API authenticates with the shared `auth_token` and drives a dedicated character (`testbot_objnum` , auto-provisioned if unset — see below). Keep `api.host` at `127.0.0.1` unless you have a specific reason to expose it, and always set a strong token.

Port selection and discovery

You do not assign ports by hand — neither the game's nor the API's. Each starts at its configured port (6770 and 7778) and, if that is taken, walks upward to the first free one, so every database is launched with the same command:

```
./mm mm.db           # game on 6770, API on 7778  
./mm scratch.db      # game on 6771, API on 7779 — nothing to coordinate
```

Each move is logged (`Port 6770 in use; auto-selected 6771` , `API port 7778 in use; auto-selected 7779`), and the pair that won is written to a **discovery file**:

```
{ "host": "127.0.0.1", "port": 7779, "pid": 38506,  
  "database": "/Users/you/megamoo/mm.db", "auth_required": true,  
  "game_port": 6771 }
```

`port` is the API's; `game_port` is the telnet port a MUD client connects to. Local tooling reads this instead of hard-coding a port — the MCP bridge does so automatically (see below). The file is removed on clean shutdown, and one left behind by a crash is ignored because its `pid` no longer names a live process. `server_status` also reports the live `api_port` and `database` .

Where it is written depends on how the server was started. `megamoo.py` puts it beside the database (`mm.db` → `mm.db.api.json`); `./mm` redirects it to `~/.megamoo/run/` so that tooling can see servers started from *other* checkouts too, under a filename built from the database's full path. The bridge searches both places.

Settings, per section: `network.auto_port` / `network.port_scan_limit` for the game listener and `api.auto_port` / `api.port_scan_limit` for the API (default 50 ports each); `api.info_path` overrides the discovery file's location, and `"-"` disables it.

Naming a port opts out of the search on that listener — `--port 6777` or `--api-port 7900` mean you want *that* port, so a conflict is a startup error (non-zero exit) rather than a silent move elsewhere.

The MCP integration

MegaMOO ships an [MCP](#) bridge (`tools/megamoo_mcp.py`) so an MCP client — such as Claude Code — can drive the running game directly: run commands as a bot character, inspect and edit live objects and verbs, search, and read logs, without touching the database by hand.

```
graph LR
    CC[Claude Code] -- stdio --> MB["tools/megamoo_mcp.py  
(MCP bridge)"]
    MB -- "TCP/JSON" --> AS["ApiServer (port 7778)  
inside running MegaMOO"]
    MB -- reads --> L["megamoo.log (works even when server is down)"]
```

The bridge is a standalone FastMCP process; the game server gains no new dependencies.

One-time setup

```
pip install mcp
./mm mm.db
claude mcp add megamoo -- python3 ~/megamoo/tools/megamoo_mcp.py
```

No token in the registration: the bridge falls back to `~/.megamoo/token` , the same file `./mm` launches every database with, so rotating it needs no config edit. Pass `-e MEGAMOO_API_TOKEN=<token>` if you'd rather set it explicitly — the environment wins.

Tools

Game tools (require the server running):

Tool	Description
<code>run_command</code>	Execute a game command as the bot character; returns its output (color stripped).
<code>disconnect_testbot</code>	Cleanly unpuppet the bot via the normal <code>on_unpuppet</code> path.
<code>get_object</code>	Full object info: name, parent, flags, properties, verbs.
<code>get_location</code> / <code>list_contents</code>	An object's location / a room or container's contents.
<code>list_verbs</code> / <code>get_verb</code> / <code>set_verb</code> / <code>delete_verb</code>	Read and write verb source live.
<code>list_properties</code> / <code>get_property</code> / <code>set_property</code>	Read and write properties.
<code>eval_code</code>	Evaluate arbitrary Python in the verb context.
<code>search_verbs</code> / <code>search_objects</code>	Full-text search across verb source / objects by name or description.

Disk tools (work even when the server is down):

Tool	Description
<code>tail_log</code>	Last N lines of <code>megamoo.log</code> , with an optional regex filter.
<code>server_status</code>	API reachability; if up, uptime, player count, and the database/port in use.
<code>list_servers</code>	Every running server: database, telnet port, API port.
<code>use_database</code>	Point the bridge at one of them for the rest of the session.

Working with several databases

Nothing is pinned to one database. With more than one server up, ask the assistant which are running and which to work on:

you: what MegaMOO servers are running? **assistant:** (`list_servers`) two — `mm.db` (game 6770, API 7778) and `scratch.db` (game 6771, API 7779). **you:** switch to scratch **assistant:** (`use_database("scratch")`) now on `/Users/you/megamoo/scratch.db`.

`use_database` accepts `mm`, `mm.db`, or a full path, takes effect on the next call, and needs no restart of the client or the bridge. An empty string returns it to auto-discovery.

Finding the server

The bridge resolves the API address on every (re)connect, so it follows a server that restarted onto a different port without restarting the MCP process:

1. `MEGAMOO_API_PORT` — an explicit pin, skipping discovery.
2. A database selected this session with `use_database`.
3. `MEGAMOO_API_INFO` — a named discovery file.
4. `MEGAMOO_DB` — that database's discovery file (`<db>.api.json`).
5. The only live discovery file in `~/.megamoo/run/` or the repo root — the everyday case: one server up, on whatever port it selected.
6. Otherwise port 7778.

With several servers advertising and no hint, the bridge refuses to guess and lists what it found.

`MEGAMOO_DB` is still there for a permanent default, but `use_database` is the everyday answer — it costs a sentence instead of a config edit.

Notes

- The bot character is **auto-created on first run_command** if no character of the configured name exists under `#5 ICharacter`. To pin a specific character, set `api.testbot_objnum` before starting the server.
- The integration test suite (`python3 -m pytest tests/`) boots its own server on separate ports against a scratch copy of the database, so it can run alongside a dev server. Two caveats: the test subprocess appends to the shared `megamoo.log`, and the database snapshot is a plain file copy (could tear under heavy live writes). When in doubt, stop the server or run only the unit tests: `python3 -m pytest tests/ --ignore=tests/test_integration_api.py`.

Security checklist

Before exposing a server beyond your own machine:

1. **Change the wizard password** from the one `megamoo init` generated, or at least confirm you still have it (see [First login](#)).
2. **Keep the JSON API local.** Leave `api.host` at `127.0.0.1` and set a strong `api.auth_token`; only widen it deliberately.
3. **Enable TLS** (`tls_port` with `tls_cert / tls_key`, or the `--tls-port` flags) if the server is reachable over the internet. It is an additional listener: the plain port keeps working, because `telnet` cannot speak TLS. Passwords cross the plain port in cleartext.
4. **Decide where TLS terminates for the browser client.** Either the engine holds the certificate (`--web-tls`), or a reverse proxy does and the engine serves plain HTTP behind it. The proxy is usually the better answer: the engine builds its SSL context once at boot, so a renewed certificate only takes effect on a restart — and a restart disconnects every player. If you use a proxy, add `--web-host 127.0.0.1` so the plain port is not independently reachable, and don't rely on a firewall rule to hide it.
5. **Check the origins if you serve the client from a second domain.** The default is same-origin only, which is what a browser loading the client from this server needs. `--web-origins` adds origins for a separate front end; `*` accepts any origin and should not survive contact with a public deployment.
6. **Turn off debug mode** in production so internal state isn't exposed.
7. **Right-size** `max_connections` for your hardware, and review the per-IP connection rate limit.
8. **Keep** `backup_on_start` **on** (default) and set up **external backups** of the database.
9. **Grant the minimum auth tier.** Give staff the lowest `gm` level that lets them do their job (see [the permission ladder](#)), and keep the set of `gm5` accounts small.

Next: [Engine Systems](#) — hooks, the ticker, and the effects system; then [The Prototype Library](#).

05 — Engine Systems

Three runtime systems sit between “a verb runs once when typed” and “the world has ongoing, reactive behavior”: **hooks** (verbs that fire on engine lifecycle events), the **ticker** (verbs that fire on a schedule), and **effects** (a higher-level timed buff/debuff layer built on the ticker). All three are available to any verb author; none require engine changes.

- [Hooks](#)
- [The ticker system](#)
- [The effects system](#)

Hooks

A **hook** is a named lifecycle event the engine fires while performing an action — moving an object, recycling it, a player connecting. Verb code can intercept the event by defining a verb with the right name on the affected object (or any ancestor). For *cancellable* hooks, the verb can return `False` to veto the action. The system lives in `moo/hooks.py`.

How resolution works

Each hook point maps a logical name to a list of **verb-name aliases**. When the engine calls `fire_hook("before_move", obj, args)`, it searches `obj` and its inheritance chain for a verb matching any of that hook’s aliases; the first match runs. If none exists, nothing happens (hooks are opt-in). So to react to an event, you just add a verb with one of the alias names to the object that should react.

```
fire_hook(hook_point, obj, args_str='') # returns the verb's result (or None)
```

For a cancellable hook, `fire_hook` returns `False` when the hook verb returned `False`, and the engine aborts the pending action.

The hook registry

These hook points are registered at startup. The **alias** column lists the verb names you can define to handle the event; the first is canonical.

Hook	Fired on	Cancellable	Aliases (verb names you define)
<code>before_move</code>	the mover	✓	<code>at_before_move</code> , <code>at_pre_move</code>
<code>before_leave</code>	old location	✓	<code>at_before_leave</code>
<code>before_enter</code>	destination	✓	<code>at_before_enter</code>
<code>after_move</code>	the mover	—	<code>at_after_move</code> , <code>at_post_move</code>
<code>after_leave</code>	old location	—	<code>at_after_leave</code> , <code>exit_func</code>
<code>after_enter</code>	destination	—	<code>at_after_enter</code> , <code>enter_func</code>
<code>object_creation</code>	the new object	—	<code>at_object_creation</code>
<code>before_recycle</code>	object being deleted	✓	<code>at_before_recycle</code>
<code>object_delete</code>	location of the deleted object	—	<code>at_object_delete</code>
<code>before_reparent</code>	object being reparented	✓	<code>at_before_reparent</code> , <code>at_pre_reparent</code>
<code>after_reparent</code>	object after reparenting	—	<code>at_after_reparent</code> , <code>at_post_reparent</code> (args = old parent #)
<code>on_disconnect</code>	player object	—	<code>at_disconnect</code> , <code>on_disconnect</code>
<code>on_puppet</code>	character	—	<code>at_puppet</code> , <code>on_puppet</code> (fires after move to <code>last_location</code>)
<code>on_unpuppet</code>	character	—	<code>at_unpuppet</code> , <code>on_unpuppet</code> (fires before storage in #2)

Note the `enter_func` / `exit_func` aliases on `after_enter` / `after_leave` — that’s how containers and rooms run arrival/departure logic. The `at_post_move` alias on `after_move` is the standard “auto-look on arrival” hook (defined on #3 so every character inherits it).

A movement is a sequence of hooks

`move()` doesn’t fire one hook — it fires a sequence, any of whose *before* hooks can cancel the whole move:

1. `before_move` on the mover (cancellable)

2. `before_leave` on the old location (cancellable)
3. `before_enter` on the destination (cancellable)
4. — the move happens —
5. `after_move` on the mover, `after_leave` on the old location, `after_enter` on the destination

Examples

Block movement while a condition holds (cancellable before-hook):

```
# verb 'at_before_move' on a character
if getattr(pobj, 'rooted', 0):
    pObj.msg("You can't move – your feet are rooted to the floor!")
    result = False      # cancels the move
```

React after a player arrives (informational after-hook):

```
# verb 'at_after_enter' (or 'enter_func') on a room
this.msg_room("%S arrives.", exclude=[pobj], sub=pobj)
```

Registering your own hook

Game code can add hook points at runtime:

```
register_hook(name, aliases, cancellable=False, description="")
list_hooks()           # all registered hooks + metadata
is_cancellable(name)   # True for before-hooks
```

`register_hook("before_say", ["at_before_say"], cancellable=True, ...)` then lets any object veto or rewrite speech by defining `at_before_say`.

The ticker system

The ticker is the engine's heartbeat: "call this verb on this object every N seconds." It is the timing backbone for the effects system, round-time countdowns, NPC behavior loops, and any recurring behavior. It lives in `moo/ticker.py`.

API

```
ticker_add(interval, verb_name, obj, idstring='')    # subscribe
ticker_remove(obj, idstring='')                     # remove one subscription
ticker_remove_all(obj)                              # remove all for an object
ticker_list(obj=None)                               # list subscriptions
```

Parameter	Meaning
<code>interval</code>	Seconds between fires (float).
<code>verb_name</code>	The verb to call on <code>obj</code> each tick.
<code>obj</code>	Target object (a <code>M000bject</code> or an objnum).
<code>idstring</code>	Unique label for this subscription on that object.

The `idstring` must be unique per object; re-adding with the same object + `idstring` **replaces** the existing subscription. Conventionally it embeds the objnum, e.g.:

```
ticker_add(1, '_td_rt', pobj, f'_td_rt_{pobj.objnum}') # tick down round-time every
second
```

Resolution and threading

The ticker has **1-second resolution**: the event loop checks once per second for due subscriptions. Tick callbacks run on the **same single verb-execution worker** as player commands (see [Architecture](#)), so they never run concurrently with a command or with each other — no locking required inside tick verbs.

Persistence

Subscriptions are stored in the `tickers` **table of the SQLite database**, so they survive restarts. On startup every subscription is reloaded and its `next_fire` is recalculated from the current time. (The older one-file-per-object builds wrote a `tickers.json`; the current SQLite engine uses the table.)

Self-cancelling tickers

A common pattern: a tick verb that counts a property down and removes its own subscription when it reaches zero. The round-time / status countdown machinery on `#1` drives many timers (`rt`, and status/condition counters) through a single configurable tick-down verb, each removing itself when it expires — so idle characters carry no live tickers.

The effects system

Effects are timed, repeating, optionally-stacking modifiers — poison ticking damage, regeneration restoring it, intoxication wearing off — built on top of the ticker. The whole system is one utility object: `#53`, reached as `$eu`. Source: `moo/effects.py`, with the callable verbs in `moo verbs/53/`.

Write `$eu`, not `eu`. There is no bare `eu` name in the verb namespace. `$eu` is a [named object constant](#): the preprocessor rewrites it to `db.get_object(0).eu`, which resolves to #53. The namespace does also carry the manager module as `_effects`, but `$eu` is the form to use.

API

```
$eu.trigger(pobj, name, ticks, interval, *args, **kwargs)
$eu.trigger_all(pobj, effects_list)
$eu.cancel(pobj, name=None)
$eu.list_active(pobj)
```

Method	Purpose
<code>trigger(pobj, name, ticks, interval, *args, **kwargs)</code>	Apply an effect that fires <code>ticks</code> times, <code>interval</code> seconds apart. Extra args are forwarded to the callback.
<code>trigger_all(pobj, effects_list)</code>	Apply several at once; each item is <code>(name, ticks, interval, *extra)</code> .
<code>cancel(pobj, name=None)</code>	Cancel one named effect, or all effects on <code>pobj</code> if <code>name</code> is omitted.
<code>list_active(pobj)</code>	Return the active effects on <code>pobj</code> (<code>name</code> , <code>remaining</code> , <code>tick</code> , <code>interval</code>).

```
$eu.trigger(pobj, "poison", 5, 3, damage=10)           # 5 ticks, every 3s, 10 dmg each
$eu.trigger_all(pobj, [("poison", 5, 3, 10), ("regen", 10, 6)])
```

Effect callbacks

Each effect named `X` is implemented by a verb `do_X` on #53. When the effect fires, the engine calls `do_X` with:

Variable	Meaning
<code>pobj</code>	The target the effect is on.
<code>tick</code>	Current fire count (1-based).
<code>remaining</code>	Ticks left after this fire.

```
# do_intoxicate on #53
if tick == 1:
    pobj.msg("Your vision begins to blur...")
pobj.msg("The world sways around you." if remaining else "Your head begins to clear.")
```

Adding a new effect type is therefore just: write a `do_<name>` verb on #53, then `$eu.trigger(pobj, "<name>", ...)` from anywhere.

Stacking and persistence

Calling `trigger` again with the same target, name, interval, and args **stacks** — it adds to the remaining ticks rather than creating a duplicate. Active effects live in the `fx_registry` property on #53, so (being a property on a database object) they persist across restarts.

The dispatcher

A hidden `_tick` verb on #53 is registered as a 1-second ticker. Each second it scans `fx_registry`, fires every effect whose `next_fire` is due, and — when the registry empties — removes its own ticker so an idle world spends no cycles on effects. This is the effects system eating its own dog food: it's just a ticker plus a dict.

Next: [The Prototype Library](#) — the shippable base objects (rooms, exits, containers, furniture, wearables, consumables) and their properties.

06 — The Prototype Library

The shipped database (#0 through #54, plus #90/#91) includes a library of base prototypes you build the world from: room types, the five exit kinds, containers, furniture, wearables, and consumables. This chapter is the reference for those prototypes — the properties they define and the `<verb>_` hooks they respond to.

These are **engine prototypes**, not game content: they model space, movement, containment, clothing layers, and consumption generically. A specific game adds its own subclasses and content on top, documented separately.

Property names below are the load-bearing ones used by the engine's own verbs. For the authoritative, current list on any prototype, use `+props/all #N` in-game (and `+verbs/all #N` for verbs). See [Building Worlds](#).

- [Rooms and the compass](#)
- [Exits](#)
- [Containers and spatial storage](#)
- [Furniture](#)
- [Wearables](#)
- [Consumables](#)

Rooms and the compass

Rooms descend from **#15 BaseRoom**, with two concrete types: **#16 OCRoom** (out-of-character) and **#17 ICRoom** (in-character). All player commands live on these two — see [where verbs live](#).

Room properties

Property	Meaning
<code>directions</code>	Per-direction exit slots for the 12-direction compass.
<code>dexits</code>	Virtual exit data — lightweight direction links stored as room data rather than as separate exit objects.
<code>exits</code>	The object-backed exits located in this room.
<code>obvexits</code>	The exits shown in the room's "obvious exits" line.
<code>plist</code>	Players currently in the room.

The 12-direction compass

Directions are defined in `moo/globals.py`. `DNAMES` holds the canonical token for each of the 12 direction slots, in index order 0–11. Note that these tokens are not uniformly short or long: the first four are spelled out, the rest are abbreviated.

Idx	DNAMES token	Also accepted	Reverse (RETEXIT)
0	north	n	south
1	south	s	north
2	east	e	west
3	west	w	east
4	ne	northeast	sw
5	nw	northwest	se
6	se	southeast	nw
7	sw	southwest	ne
8	u	up	d
9	d	down	u
10	o	out	—
11	in	—	—

Players may type either form (`n` or `north` , `u` or `up`); `DALIASES` pairs them and `DIRECTIONS` is the flat list of every accepted token. The `RETEXIT` table maps each direction to its opposite and is what `@dig / @open` use to auto-create the **return exit** — it has no entry for `o` or `in` , so passages through those are created one-way unless you open both ends.

Room verbs

`look` / `look_here` (render the room), `go` and the directional verbs (movement), `match_exit` (resolve a direction or keyword to an exit), and `gmove` (the generic move worker exits call back into).

Exits

All exits descend from **#20 BaseExit**. The five concrete kinds and their builder commands (full details in [Building Worlds](#)):

#	Prototype	Built with	Traversed by
#21	DirectionalExit	@open / @vopen	a compass direction
#22	GoExit	@gopen	go <name>
#23	ClosableGoExit	@dopen	go <name> (must be open)
#24	ClimbableExit	@copen	climb <name>
#25	JumpableExit	@jopen	jump <name>

Exit properties

Property	Meaning
source	The room the exit leads from.
destination	The room the exit leads to.
success / osuccess	Message to the mover / to onlookers on a successful traversal.
drop / odrop	Message to the mover on arrival / to onlookers in the destination.
mode / omode	First- / third-person movement verb ("walk" / "walks").
closed	Whether a closable exit is currently closed.
locked / latched	Lock / latch state on closable exits.

Exit message tokens

Exit messages substitute these tokens (defaults in `globals.py` as `ESUCC` / `E0SUCC` / `E0DROP`):

Token	Renders
%MODE	first-person movement verb (from <code>mode</code>)
%OMODE	third-person movement verb (from <code>omode</code>)
%S	the mover's styled display name
%1 / %dir	the direction name

```

success: "You %MODE %1."      → "You walk north."
osuccess: "%S %OMODE %1."    → "Alice walks north."
odrop:   "%S %OMODE in from the %1." → "Alice walks in from the south."

```

These are distinct from the emit tokens used by `msg` / `msg_room` (see [Writing Verbs](#)); exit messages use the `%MODE` / `%1` set above.

Containers and spatial storage

Spatial storage is split across two prototypes:

- **#11 (object)** defines the four **spatial relations** — `in`, `on`, `under`, `behind` — that any object supports, via the `<rel>_put` / `<rel>_get` / `<rel>_look` verb family. So you can `put vase on statue` or `look under rug` even when the target isn't a "container."
- **#26 (BaseContainer)** adds an enclosed interior with capacity limits and open/closed state. Concrete types: **#27 Chest**, **#28 Cup**, **#29 Box**.

Spatial prepositions

<code>put book in chest</code>	→ <code>in_contents</code>	<code>look in chest</code>	→ shows <code>in_contents</code>
<code>put vase on chest</code>	→ <code>on_contents</code>	<code>look on chest</code>	→ shows <code>on_contents</code>
<code>put key under chest</code>	→ <code>under_contents</code>	<code>look under chest</code>	→ shows <code>under_contents</code>
<code>put note behind chest</code>	→ <code>behind_contents</code>	<code>look behind chest</code>	→ shows <code>behind_contents</code>
<code>get book from chest</code>	→ removes from <code>in_contents</code>		

Container properties

Property	Meaning
<code>open</code>	Whether the container is currently open (gates <code>in_*</code> access).
<code>in_contents</code>	Objects inside. (<code>on_</code> / <code>under_</code> / <code>behind_contents</code> exist on #11 for the other relations.)
<code>max_vol</code>	Maximum total volume the interior holds.
<code>max_items_in</code>	Maximum item count inside.
<code>max_weight_in</code>	Maximum total weight inside.
<code>pass_locks</code>	Lock-bypass configuration for the interior.

Container verbs

`in_put` / `in_get` / `in_look` (the interior operations, which enforce open state and the capacity limits), and `look_` (how the container renders when examined). The `enter_func` / `exit_func` hooks (the `after_enter` / `after_leave` aliases — see [Engine Systems](#)) run when something enters or leaves a container you can step into.

Furniture

Furniture descends from **#30 BaseFurniture**: things players sit on, lie on, or gather at. Concrete types include **#31 Chair**, **#32 Bed**, **#33 Table**, **#34 Bench**.

Furniture properties

Property	Meaning
<code>seats</code>	How many players the piece holds.
<code>sitters</code>	Players currently using it.
<code>sit_prep</code>	The preposition used — <code>on</code> (chair/bench), <code>at</code> (table), <code>in</code> (e.g. a tub).

The position system

A character's `position` property is an integer:

Value	Position
0	standing
6	sitting
7–9	lying (varying poses)

Sitting or lying on furniture sets the value and adds the character to the piece's `sitters`; `stand` clears it.

Furniture verbs

`sit_`, `lay_` (the per-object hooks the room's `sit` / `lay` verbs dispatch to, which check capacity and `sit_prep`), `look_` (renders the piece and who's on it), and `tt` — **table talk**, which speaks only to others at the same piece of furniture.

Wearables

Wearables descend from **#35 BaseWearable** (itself a child of #12 item). The shipped clothing prototypes are **#36 Shirt**, **#37 Pants**, **#38 Shoes**, **#39 Hat**, and **#40–#49 BlankClothing** (blank templates to clone and configure).

Body slots and the layer system

A wearable's `wear_pos` is a list of `[position, size]` pairs naming the body slots it occupies and how it layers there. Layering is **size-based**:

- **Size 0 — accessories.** Don't participate in layering; multiple can share a slot (rings, bracelets).
- **Size 1+ — layers.** A larger size goes over a smaller one in the same slot; you can't put a smaller garment over a larger one. Removing an inner layer is blocked while something larger covers it.

Bilateral (left/right) slots are encoded by position values over 100: the base slot is `pos % 100` (the **right** side) and the **left** is `base - 1`, so a single `wear_pos` entry can describe a left-or-right item.

Configure all of this in-game with `@wearpos` (a game/staff command — see its note in [Building Worlds](#)).

Wearable properties

Property	Meaning
<code>wear_pos</code>	List of <code>[position, size]</code> pairs — where it's worn and how it layers.
<code>worn</code>	Whether it's currently worn.
<code>size</code>	The layering size of the garment.
<code>visible</code>	Whether it's visible when worn.
<code>layer_flex</code>	Per-slot flags allowing flexible layering order.

(A character tracks what it has on via its own `wearing` property.)

Wearable verbs

`wear_` and `remove_` (the per-object hooks the room's `wear / remove` dispatch to — they check slot availability, layer order, and what's on top), plus `items_on_top` / `items_under` (the helpers that report layering conflicts and so gate removal).

Consumables

Two prototypes, both children of #10: **#90 BaseEdible** (food) and **#91 BaseDrinkable** (beverages). Both support multiple uses, effects on consumption, and custom messages.

Properties

Property	Meaning
<code>uses</code>	Bites / sips remaining.
<code>prepared</code>	Whether the item is ready to consume.
<code>effects</code>	Effect(s) applied to the consumer on each use (via the effects system).
<code>eemits</code>	Messages emitted to the room on each use.
<code>rtdice</code>	Dice parameters for randomized effect application.
<code>finish</code> / <code>ofinish</code>	Message to the consumer / to onlookers on the final use.
<code>liquid</code>	Marker on #91 identifying it as a beverage (used by cup containers).

Verbs

`eat_` (on #90), `drink_` and `sip` (on #91): each decrements `uses`, applies `effects`, shows the `eemits`, and on the last use shows `finish` / `ofinish`. They are the per-object hooks the room's `eat` / `drink` / `sip` verbs dispatch to.

```
@make #90
@name <obj> = loaf of bread
@adprop <obj>.uses = 5
@adprop <obj>.prepared = True
@adprop <obj>.finish = You eat the last of the bread.
@adprop <obj>.ofinish = %S eats the last of the bread.
```

(Property values are set with `@adprop` — there is no separate `@set`; on a fresh instance these create local overrides of the inherited defaults. `@adprop` evaluates the right-hand side as a Python expression where it can, so `5` and `True` become an int and a bool, while unparseable text is stored as a string.)

Back to the [index](#).

07 — The Help System

The in-game `help` command is the player's and builder's window into the world. This chapter documents how it resolves a request, where help content lives, and how to author it — both the curated topic library and the self-documenting verb docstrings that make every command explain itself.

The whole system is one verb — `moo verbs/15/help.py`, defined on `#15` (`BaseRoom`) so it is inherited by every room, OOC and IC alike. There is no separate help engine, no help compiler, and no help database table. Help content is just **object state**: string and dict properties on one object, plus the docstrings already attached to verbs. That is the defining idea of the subsystem — *help is data you set, not code you write*.

Two sources of truth

The `help` verb draws from exactly two places:

Source	What it is	Who authors it
The topic store — <code>#34</code> (<code>help_utils</code>)	Free-form articles: lore, mechanics, policy, “how to play” prose. Each topic is a property.	Builders / staff (<code>#34</code> is owned by the <code>#100</code> wizard)
Verb docstrings	The first triple-quoted block of any verb in scope. Documents commands as a side effect of writing them.	Whoever wrote the verb

The split is deliberate. **Commands document themselves** — the same docstring a programmer reads in the source is the help a player reads in the game, so the two can never drift apart.

Everything that isn't a command — combat rules, the chargen walkthrough, world history, the rules of conduct — lives as editable prose on `#34`, where staff can revise it live without touching a verb.

How a request resolves

`help` runs a fixed resolution ladder and stops at the first rung that matches. Understanding the order is the key to both using and authoring help.

help with no argument — the index

Two lists are printed:

1. **Help Topics** — every own property on #34 except the bookkeeping props `name`, `name_mod_list`, and `description`, and anything whose name begins with an underscore, which is how this object marks its own configuration rather than a topic. Only the object's *own* properties count (`properties_list(include_inherited=False)`), so nothing leaks in from #34's parent.
2. **Command Help** — the visible verbs reachable from where the player is *standing right now*: the verbs on `pobj`, on `pobj.location`, and on every object in the room's contents. A verb is listed only if it has a docstring and the player is allowed to see it (see [Visibility](#)).

Because the command list is built from the current scope, **help is context-sensitive**: the commands a player sees offered are the commands actually available in that room, on those objects. Walk somewhere else and the list changes.

help <topic> — the lookup ladder

For a non-empty argument that does not start with `#`, the verb tries, in order:

Rung	Match	Result
1	Argument equals an own property name on #34 (case-insensitive)	If the value is a string , print it. If it's a dict , treat it as a category and print its sorted keys (its sub-topics).
2	Argument equals a key inside any dict property on #34	Print that sub-topic's text, headed <code>category > key</code> .
3	Argument matches a verb name in scope (<code>pobj</code> , <code>location</code> , <code>contents</code>)	Extract and print the verb's docstring. Respects auth gating.
—	Nothing matched	There's no help for that.

So a single word like `combat` can be satisfied by a standalone article, by a sub-topic nested inside a category, or by a command of that name — whichever the ladder reaches first. Author topics with that precedence in mind: a #34 property name shadows a verb of the same name.

help #<object> and help #<object>.<verb> — introspection

A leading `#` switches into **object introspection**, gated to `gm3` and above (everyone else gets `There's no help for that.`, so the feature is invisible to players):

- `help #42` — prints #42's `help_text` property, if set. This is a free-form string property you can attach to *any* object to document what it is and how to use it — handy for a complex machine, NPC, or builder fixture.

- `help #42.go` — finds the verb `go` **defined on #42 itself** (not inherited) and prints its docstring, after checking the viewer's auth against the verb's `auth` level. Use it to read the help for a specific object's override without standing next to it.

Authoring topic articles

Topics are properties on `#34`. Add them in-game with the builder toolkit or from a wizard session.

A standalone article (string property)

`@adprop` creates the topic and takes its text in one go. `@set` is for revising it afterwards – it writes a property that already exists and cannot create one:

```
@adprop #34.rules = "Be excellent to each other. No harassment, no cheating..."
@set #34.rules = "Be excellent. No cheating. No spoilers in public channels."
```

`help rules` now prints that text. The property name is the topic name, so keep it a single lowercase word or short token — that is exactly what the player types.

For multi-line prose, set it from the verb editor or via `set_property` so you can embed newlines and color codes (e.g. `&<245>` for dim headers, `&n` to reset — see [Architecture](#) and the color conventions used elsewhere in the world):

```
set_property(34, 'combat',
  "&<245>Combat&n\n"
  "Attacks are resolved on a roundtime clock. Type 'attack <target>'...\n"
  "See also: help wounds, help parry")
```

A category (dict property)

A property whose value is a **dict** becomes a *category*: an index whose keys are sub-topics. The category name lists its contents; each key is reachable directly.

```
set_property(34, 'magic', {
  'spells': "Casting consumes mana. Type 'cast <spell> at <target>'...",
  'mana':   "Mana regenerates while resting. Your pool scales with...",
  'schools': "There are four schools: evocation, abjuration...",
})
```

Now:

- `help magic` → lists `mana`, `schools`, `spells` (sorted keys — rung 1, dict branch).
- `help spells` → prints the spells text, headed `magic > spells` (rung 2).

Note the asymmetry: the **category name** (`magic`) appears in the no-argument *Help Topics* index, but the **sub-topic keys** do not — they are discovered by reading the category or by knowing the word. Categories keep the top-level index short while still making every sub-topic directly addressable.

Staff-only topics

A topic can require an auth level. `_topic_auth` on `#34` maps a topic name to the level needed to read it, and a topic not named there is public:

```
@set #34._topic_auth = {"bmatch": 3, "pmatch": 3, "matching": 3}
```

Those three are the shipped example: they document the two matchers a person chooses between *while writing a verb*, which is a decision a player never makes. Below the level, the topic is absent from the *Help Topics* index **and** `help <topic>` answers “There’s no help for that” — the same answer as for a topic that does not exist, because “you may not read this” tells someone fishing that there is something there to read. Hiding a topic from the index while still printing it on request is not a gate.

The gate applies to the topic, so a category’s sub-topics go with it.

What never becomes a topic

`name`, `name_mod_list`, and `description` on `#34` are filtered out of the topic index, so you can name and describe the help object normally without those showing up as bogus topics.

Authoring command help

A command documents itself through its **docstring** — the first `"""..."""` (or `'...'...`) block in the verb’s source. The help verb extracts the text between the opening and closing triple-quotes, strips it, and prints it verbatim. No annotation, registration, or separate help entry is required: write the docstring and the command is documented.

The house style — visible in nearly every shipped verb — is a one-line summary, a `Usage:` line, and optional examples:

```
"""
Jump across a gap or obstacle.

Usage: jump <exit>
"""
```

`help jump` (while standing where `jump` is available) prints that block. Because the docstring is the same text the next programmer reads, keeping help accurate is the same act as keeping the source readable.

Visibility: hidden and gated verbs

The help verb honors two verb attributes so help never advertises a command the player can't use or shouldn't see:

- **hidden** — a verb flagged hidden (e.g. by `@hideverb`) is omitted from the no-argument *Command Help* index. It can still be looked up by name if you know it, but it isn't broadcast.
- **auth** — if a verb declares a minimum auth level, players below that level see neither the verb in the index nor its docstring on direct lookup; the lookup falls through as though the verb weren't there. Staff-only commands therefore stay invisible to players in the help system automatically. (`auth_level(pobj)` supplies the viewer's level; the permission ladder is in [Operations](#).)

This is why command help is trustworthy: the list a player sees is, by construction, the list of commands available *and* permitted to them, in their current location.

Worked example: adding a “factions” topic

A builder wants `help factions` to exist as a category with three entries.

```
# from a wizard eval / verb editor
set_property(34, 'factions', {
    'guild':  "The Tinkers' Guild controls trade in the eastern wards...",
    'crown':  "The Crown's agents enforce the King's law...",
    'free':   "The Free Companies answer to coin and little else...",
})
```

Verification, the way a player would experience it:

```
help          -> "factions" now appears under Help Topics
help factions -> lists: crown, free, guild
help crown    -> prints the crown text, headed "factions > crown"
```

No restart and no verb edit — the topic is live the moment the property is set, because help content is object state read on every invocation.

Quick reference

Command	Who	Effect
<code>help</code>	all	Index: topic names on <code>#34</code> + visible commands in scope
<code>help <topic></code>	all	Article (string), category index (dict), sub-topic, or command docstring — first match wins
<code>help #<obj></code>	gm3+	Print the object's <code>help_text</code> property
<code>help #<obj>.<verb></code>	gm3+	Print the docstring of that verb as defined on that object

To author...	Do this
A standalone article	Set a string property on <code>#34</code> : <code>@set #34.<topic> = "..."</code>
A category of sub-topics	Set a dict property on <code>#34</code> : <code>set_property(34, '<cat>', {...})</code>
Command help	Write a triple-quoted docstring as the first thing in the verb
Help for an object	Set a <code>help_text</code> string property on that object
Hide a command from the index	Flag the verb <code>hidden</code> (e.g. <code>@hideverb</code>)
Restrict help to staff	Give the verb an <code>auth</code> level

Design notes and gotchas

- **One object holds all articles.** `#34` (`help_utils` , owned by `#100`) is the entire topic library. Back it up with the rest of the database; there is no separate help store to manage.
- **Precedence matters.** A `#34` property name shadows a verb of the same name (rung 1 before rung 3). If you add a `look` property to `#34` , `help look` stops reaching the `look` command. Name topics so they don't collide with commands unless you intend to override them.
- **Sub-topic keys are not indexed.** They are addressable (`help <key>`) but do not appear in the no-argument list — only their category does. Mention important sub-topics in the category's own prose, or in a "See also:" line, so players can find them.
- **Command help is local.** The no-argument index reflects the current room and the objects in it. A command defined on an object in another room won't appear until the player is in scope of it. This is a feature — it surfaces what's usable here — but don't expect a global command list from `help` alone.

- **Help is read live.** Every `help` invocation re-reads `#34` and the verbs in scope, so edits to topics or docstrings take effect immediately, with no reload.
- **The login banner points here.** New connections are greeted with “*Type ‘help’ for help, or ‘quit’ to disconnect.*” (`moo/globals.py` , `moo/server.py`), making this verb the documented front door to the world.

